

Proyecto estructuras de datos

Daniel Castro, Victoria Acero, Juan Rozo.

Pontificia Universidad Javeriana, Bogotá D.C, Colombia

castro-df@javeriana.co

juan.rozot@javeriana.co

jv.acero@javeriana.co

Abstract. This paper outlines the design process of a system for managing 3D objects and performing calculations on their components, particularly vertices. Computer-Aided Design (CAD) is a field of modern design that allows for the graphic representation of objects through complex mathematical calculations. However, in this project, a simpler approach is taken, starting with the definition of four Abstract Data Types (ADTs): menu, object, face and vertex; and the creation of a diagram to illustrate their relationships. The system includes basic functions such as downloading and uploading an object, saving it to a file, generating meshes among objects and a global mesh, checking if an object is present in memory, listing all objects, and deleting specific ones. Following the implementation, an evaluation protocol was established to assess each function's behavior under various conditions, including edge cases and typical data. Finally, a report was compiled to summarize the test results, and necessary corrections were made.

Keywords: ADT, CAT, designing, testing.

1 Descripción del problema

En el campo de la computación gráfica, la representación de objetos tridimensionales es esencial para el desarrollo de entornos virtuales, simulaciones, diseño asistido por computadora (CAD), animación y videojuegos (Bender & Manfred, 2003; Filler, 2001). Los modelos tridimensionales permiten que los algoritmos de renderización reproduzcan gráficos realistas que simulan con precisión las propiedades físicas y estéticas de los objetos, sean estos reales o imaginarios. Este proceso de simulación es fundamental no solo para crear gráficos visualmente atractivos, sino también para asegurar una interacción efectiva en entornos virtuales, donde la precisión y el detalle son importantes (Bender & Manfred, 2003).

Uno de los métodos más utilizados para la representación de objetos tridimensionales es la modelación poligonal, una técnica que se basa en el uso de polígonos para definir las superficies de los objetos (Filler, 2001). Este enfoque ha ganado gran aceptación debido a su compatibilidad con el hardware gráfico contemporáneo, que está optimizado para procesar grandes cantidades de polígonos con alta eficiencia (Filler, 2001). Entre los polígonos, el triángulo se destaca como la forma más básica y utilizada; su

simplicidad, junto con sus propiedades planas y convexas, lo convierten en una opción ideal para la renderización y manipulación en sistemas gráficos modernos (Gudalo, 2012). La capacidad de los triángulos para adaptarse fácilmente a diferentes formas y superficies permite una gran flexibilidad en la modelación tridimensional (Gudalo, 2012).

Los modelos poligonales se construyen mediante la integración de vértices, aristas y caras. Un vértice es un punto específico en el espacio tridimensional, caracterizado por coordenadas x , y , z que determinan su ubicación relativa en un sistema de coordenadas. Estos puntos actúan como bloques básicos de construcción para formas y superficies más complejas, estableciendo la estructura geométrica del modelo. Las aristas son líneas que conectan pares de vértices, delineando el contorno de las caras del modelo, representando tanto la conexión física entre los vértices como la transición espacial entre las superficies. Por su parte, las caras son áreas planas delimitadas por aristas, generalmente compuestas por tres o más vértices. Estas caras desempeñan un papel crucial en la visualización poligonal, ya que son las superficies visibles sobre las que se aplican texturas y efectos de iluminación para lograr un realismo visual (Filler, 2001). Los componentes mencionados están organizados en estructuras de datos que van desde listas simples de coordenadas y conexiones hasta configuraciones más sofisticadas destinadas a facilitar manipulaciones avanzadas y mejorar la eficiencia en la renderización (Bender & Manfred, 2003; Gudalo, 2012).

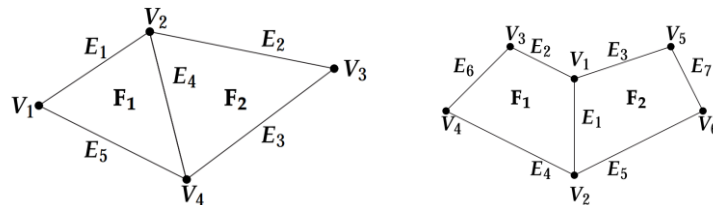


Figura 1. Modelo poligonal (Bender & Manfred, 2003)

En la representación gráfica, los puntos etiquetados con 'v' representan los vértices, las letras 'E' indican las aristas, y las 'f' denotan las caras.

En el contexto de este proyecto académico, cada malla u objeto tridimensional se estructura siguiendo un formato específico que asegura una representación sistemática adecuada para su procesamiento por software. Este formato se compone de varios elementos que se detallan a continuación:

- **Nombre de la malla:** El nombre de la malla es el identificador del objeto 3D, ubicado al inicio del archivo. Se trata de una cadena de caracteres que describe el objeto representado, y es importante que no contenga espacios para evitar problemas en el procesamiento.

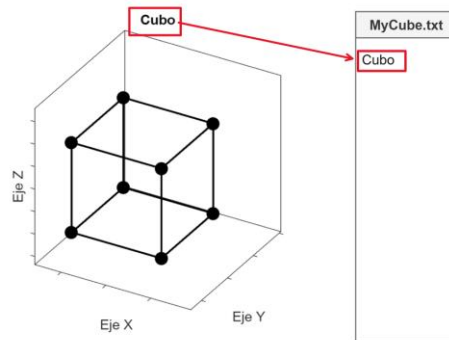


Figura 2. Nombre de malla poligonal.

- Número de vértices: La primera cifra numérica después del nombre de la malla indica la cantidad total de vértices que la componen. Este número es esencial, ya que determina cuántas líneas subsiguientes en el archivo estarán dedicadas a describir las coordenadas de estos vértices.

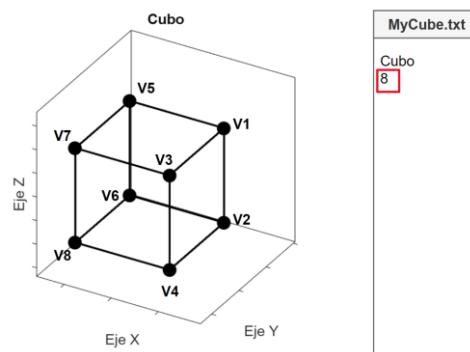


Figura 3. Número de vértices

- Coordenadas de los vértices: A continuación del número de vértices, cada línea del archivo describe un vértice en el espacio tridimensional, especificando sus coordenadas x , y y z . Estos vértices son los puntos de anclaje para las aristas y caras del objeto, y su precisión es vital para la integridad del modelo.

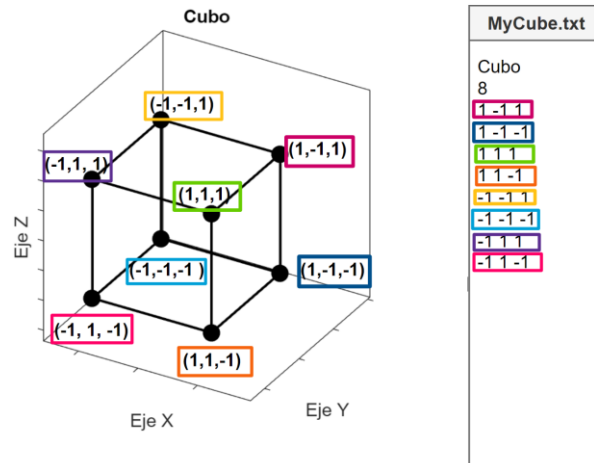


Figura 4. Coordenadas de los vértices

- Descripción de las caras: Después de enumerar todos los vértices, el archivo detalla las caras del objeto. Cada línea correspondiente a una cara comienza con un número que indica cuántos vértices conforman esa cara específica, seguido por los índices de los vértices que la definen. Los índices se basan en el orden de aparición de los vértices en el archivo, comenzando desde cero.

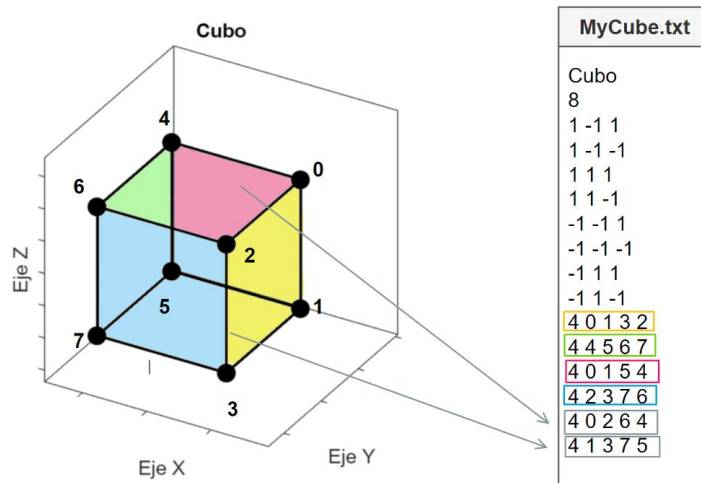


Figura 5. Descripción de las caras

- Fin de la malla: Un número -1 marca el final de la definición de la malla en el archivo, indicando que no hay más datos asociados a esa malla en particular.

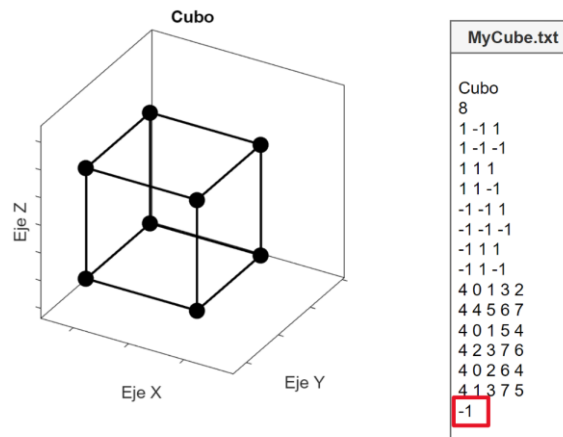


Figura 6. Fin de la malla

2 Componente 1: organización de la información

El primer componente del proyecto se enfoca en la gestión de información de objetos tridimensionales representados mediante mallas poligonales. Los comandos desarrollados en esta fase están diseñados para organizar y estructurar de manera eficiente los datos 3D, lo cual es esencial para las etapas posteriores del proyecto. Este componente establece las bases del sistema al facilitar la carga y manipulación de los elementos de las mallas, como vértices, aristas y caras, garantizando un acceso y uso óptimo de la información.

2.1 Descripción de los Comandos

Cargar <NombreArchivo.txt>

El comando cargar está diseñado para cargar un objeto 3D desde un archivo de texto en la memoria del programa. Este proceso implica varias etapas que deben cumplirse para que el comando funcione correctamente, comenzando por la apertura del archivo especificado por el usuario. El archivo debe existir y ser legible; de lo contrario, el comando finalizará inmediatamente y mostrará un mensaje indicando que el archivo no existe o es ilegible.

Una vez que el archivo ha sido abierto correctamente, el siguiente paso es leer el nombre del objeto desde la primera línea de este. Es crucial que el nombre del objeto no coincida con el de ningún objeto previamente cargado en memoria, ya que el comando está diseñado para evitar duplicados. Si el nombre ya existe, el comando se detendrá y emitirá un mensaje advirtiéndolo que el objeto ya ha sido cargado previamente. Posteriormente, él se intenta leer el número de vértices del objeto desde la segunda línea del archivo. Este número debe ser un entero válido. Si el formato de esta línea no es correcto, el proceso se interrumpe, y se informa al usuario que el número de vértices no es válido. Además, es esencial que el archivo no esté vacío o incompleto, ya que esto también provocará la terminación del comando con un mensaje de error adecuado.

Si el número de vértices se ha leído correctamente, el comando procederá a iterar sobre las siguientes líneas para extraer las coordenadas de cada vértice. Cada línea debe contener tres valores numéricos que representen las coordenadas x, y y z. Si alguna línea tiene un formato incorrecto o si faltan datos, el comando finalizará, indicando que las coordenadas de los vértices no son válidas.

Después de cargar todos los vértices, el comando comienza a leer las caras del objeto. Cada cara está definida por un conjunto de índices que referencian los vértices previamente leídos. El archivo debe contener un número válido de vértices por cara y los índices correspondientes deben ser válidos y estar dentro del rango de vértices cargados. Si algún índice es inválido o si el formato de la cara es incorrecto, el comando se detendrá y mostrará un mensaje de error indicando el problema.

Una vez que todas las caras han sido leídas correctamente, el comando procede a ensamblar el objeto 3D utilizando el nombre, los vértices y las caras leídas del archivo. Finalmente, el objeto es añadido a la lista de objetos cargados en memoria, y el comando concluye con un mensaje de éxito, confirmando que el objeto ha sido cargado correctamente desde el archivo.

Es importante resaltar que el comando cargar se debe invocar escribiendo cargar seguido del nombre del archivo de texto que contiene la información del objeto 3D. Por ejemplo, si el archivo se llama modelo3D.txt, el comando se escribiría como cargar modelo3D.txt. Es fundamental que el archivo tenga la extensión .txt y que esté ubicado en un directorio accesible para que el comando pueda procesarlo adecuadamente.

Listado

El comando listado se invoca simplemente escribiendo listado. Este comando está diseñado para mostrar al usuario una lista de todos los objetos 3D que han sido cargados en la memoria del programa hasta el momento de su ejecución.

El proceso comienza verificando si existen objetos cargados en memoria. Si no hay ningún objeto en la lista, el comando finalizará inmediatamente y mostrará un mensaje

indicando que no se ha cargado ningún objeto. Esto asegura que el usuario esté informado de que no hay datos disponibles para listar.

En caso de que haya objetos cargados, el comando procederá a contar y mostrar la cantidad total de objetos presentes en memoria. A continuación, iterará sobre cada uno de ellos, presentando una descripción en la consola. La descripción de cada objeto se obtiene utilizando un método que convierte las propiedades del objeto a una cadena de texto adecuada para su visualización. Esta cadena de texto generalmente incluye detalles como el nombre del objeto y sus características esenciales, lo que permite al usuario identificar rápidamente qué objetos están disponibles y sus atributos básicos.

Envolvente <nombreObjeto>

El comando envolvente de un objeto específico se invoca escribiendo envolvente seguido del nombre del objeto que se desea envolver. Por ejemplo, si el objeto se llama Cubo, el comando se escribiría como envolvente Cubo. Este comando tiene la finalidad de generar una caja envolvente que delimita el espacio ocupado por un objeto 3D cargado en memoria como puede ser visualizado en la figura 7.

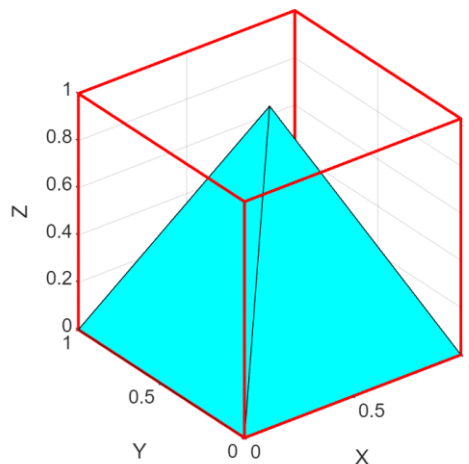


Figura 7. Caja envolvente de una pirámide

El proceso comienza buscando el objeto en la lista de objetos actualmente cargados en memoria. Si el objeto no se encuentra, el comando finalizará y mostrará un mensaje indicando de error, lo que asegura que solo se intente generar una caja envolvente para objetos válidos y existentes.

Si el objeto es encontrado, el comando procede a calcular los límites espaciales del objeto, determinando las coordenadas mínimas y máximas en los ejes X, Y y Z. Estos valores representan los puntos extremos del objeto en cada dirección, y se utilizan para

definir los vértices de la caja envolvente. Una vez determinados los límites, se crean ocho vértices que representan los ocho puntos extremos de la caja. Luego, se construyen seis caras utilizando estos vértices, de modo que la caja envolvente rodee completamente el objeto 3D. Las caras son añadidas a un nuevo objeto que representa la caja envolvente generada.

El nombre de este nuevo objeto es proporcionado automáticamente a partir del nombre del objeto original, precedido por el prefijo `env_`. Si ya existe un objeto en memoria con ese nombre, el comando agregará un número secuencial al final para garantizar que cada caja envolvente tenga un nombre único.

Finalmente, la caja envolvente es añadida a la lista de objetos cargados en memoria, y se muestra un mensaje de éxito confirmando la creación y adición de la caja, junto con su nombre. Este comando es útil para visualizar y analizar los límites espaciales de un objeto 3D, permitiendo al usuario entender mejor el espacio que ocupa el objeto dentro del entorno virtual.

Envolvente

El comando `envolvente` se invoca escribiendo simplemente `envolvente`. Este comando tiene la finalidad de generar una caja envolvente global que delimita el espacio ocupado por todos los objetos 3D que han sido cargados en memoria hasta el momento (ver figura 8).

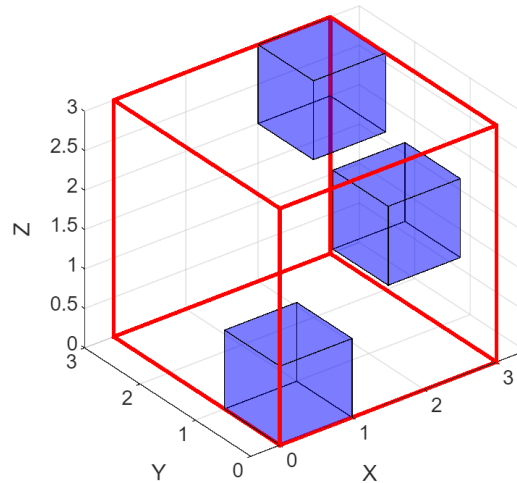


Figura 8. Caja envolvente global

El proceso comienza verificando si hay objetos cargados en memoria. Si no hay ningún objeto presente, el comando finalizará inmediatamente y mostrará un mensaje indicando que no hay objetos en memoria. Esto asegura que el comando solo se ejecute si hay datos disponibles para procesar.

Si existen objetos en memoria, el comando procederá a calcular los límites espaciales de todos los objetos combinados, determinando las coordenadas mínimas y máximas en los ejes X, Y y Z. Estos valores representan los puntos extremos más alejados en cada dirección de todos los objetos cargados, y se utilizan para definir los vértices de la caja envolvente global.

Una vez determinados los límites, se crean ocho vértices que representan los puntos extremos de la caja. A partir de estos vértices, se construyen seis caras que forman la caja envolvente, asegurando que rodee completamente todos los objetos 3D cargados en memoria.

El nombre asignado a este nuevo objeto es `caja_global`, y no se permiten nombres duplicados, dado que este nombre es específico para la caja envolvente global de todos los objetos en memoria. Una vez que la caja envolvente global ha sido construida, se añade a la lista de objetos cargados en memoria, y se muestra un mensaje de éxito confirmando la creación y adición de la caja envolvente global.

Descargar <NombreObjeto>

El comando descargar se invoca escribiendo descargar seguido del nombre del objeto que se desea eliminar de la memoria. Por ejemplo, si el objeto se llama Cubo, el comando se escribiría como descargar Cubo. Este comando está diseñado para eliminar un objeto específico que ha sido cargado en la memoria del programa, liberando así los recursos asociados a ese objeto.

El proceso comienza buscando el objeto en la lista de objetos actualmente cargados en memoria. Para encontrar el objeto, se compara el nombre del objeto proporcionado por el usuario con los nombres de los objetos cargados. Si el objeto no se encuentra, el comando finalizará y mostrará un mensaje indicando que el objeto no ha sido cargado en memoria, asegurando que el usuario esté al tanto de que el objeto no existe en el entorno actual.

Si el objeto es encontrado, el comando procede a eliminarlo de la lista de objetos cargados. Una vez eliminado, el comando muestra un mensaje de éxito, confirmando que el objeto ha sido eliminado correctamente de la memoria de trabajo. Este comando es útil cuando el usuario ya no necesita un objeto específico en la memoria y desea limpiar el entorno de trabajo.

Guardar <NombreObjeto> <NombreArchivo.txt>

El comando guardar se invoca escribiendo guardar seguido del nombre del objeto y el nombre del archivo en el que se desea guardar. Por ejemplo, si el objeto se llama Cubo

y se quiere guardar en un archivo llamado Cubo.txt, el comando se escribiría como guardar Cubo Cubo.txt. Este comando está diseñado para almacenar la representación de un objeto en un archivo de texto con las especificaciones del programa.

El proceso comienza buscando el objeto en la lista de objetos cargados en memoria. Si el objeto no se encuentra, el comando finalizará y mostrará un mensaje indicando que el objeto no ha sido cargado en memoria, lo que asegura que solo se intente guardar objetos válidos y existentes.

Si el objeto es encontrado, el comando procede a abrir o crear el archivo especificado para escritura. Si el archivo no se puede abrir, por ejemplo, debido a problemas de permisos o errores en el sistema de archivos, el comando se detendrá y mostrará un mensaje de error indicando el problema.

Una vez abierto el archivo, el comando escribe en él el nombre del objeto como primera línea. A continuación, se crea una lista de vértices únicos, que son todos los vértices del objeto sin duplicados. El siguiente paso es escribir el número total de vértices únicos en el archivo, seguido de las coordenadas x, y y z de cada uno. Luego, el comando escribe las caras del objeto, indicando cuántos vértices componen cada cara y los índices de estos vértices en la lista de vértices únicos.

Finalmente, el comando añade una línea final con el valor -1 para indicar el final y cierre del archivo. Esto completa el proceso de guardar, y el comando muestra un mensaje de éxito confirmando que la información del objeto ha sido guardada correctamente en el archivo especificado.

Salir

El comando salir se invoca simplemente escribiendo salir. Este comando está diseñado para finalizar la ejecución del programa de manera inmediata, cerrando todas las operaciones en curso. El proceso muestra un mensaje al usuario indicando que el programa se está cerrando. Este mensaje asegura que el usuario está al tanto de que el comando ha sido ejecutado y que el programa se cerrará de inmediato.

2.2 Diseño

En esta sección, se presenta el diseño del sistema, el cual será desarrollado de manera acumulativa para reflejar la evolución del proyecto en todos sus componentes. A continuación, se describen en detalle los Tipos Abstractos de Datos (TADs) utilizados en el programa, que incluyen objeto, cara, vértice, y menú.

Descripción de los Tipos Abstractos de Datos (TADs):

Vértice

El TAD vértice representa un punto en el espacio tridimensional, utilizado como base para la construcción de figuras geométricas más complejas en un sistema de modelado 3D. Este TAD maneja las coordenadas espaciales de un vértice y permite realizar operaciones sobre ellas, como la obtención de distancias y la comparación entre vértices.

Estado

- Coordenada_X: (x), numérico, indica la posición del vértice en el eje X del espacio tridimensional.
- Coordenada_Y: (y), numérico, indica la posición del vértice en el eje Y del espacio tridimensional.
- Coordenada_Z: (z), numérico, indica la posición del vértice en el eje Z del espacio tridimensional.

Interfaz

- Vertice(): Vertice, crea un vértice con valores por defecto.
- Vertice(x, y, z): Vertice, crea un vértice en la posición especificada por las coordenadas x, y, z.
 - Pre: x, y, z deben ser valores numéricos.
- getX(): x, retorna el valor de la coordenada x del vértice.
 - Pre: El vértice debe estar definido.
- getY(): y, retorna el valor de la coordenada y del vértice.
 - Pre: El vértice debe estar definido.
- getZ(): z, retorna el valor de la coordenada z del vértice.
 - Pre: El vértice debe estar definido.
- setX(x), establece un nuevo valor para la coordenada x del vértice.
 - Pre: x debe ser un valor numérico.
- setY(y), establece un nuevo valor para la coordenada y del vértice.
 - Pre: y debe ser un valor numérico.
- setZ(z), establece un nuevo valor para la coordenada z del vértice.
 - Pre: z debe ser un valor numérico.
- distancia(otro): distancia, calcula y retorna la distancia entre el vértice analizado y otro vértice dado.
 - Pre: Ambos vértices deben estar definidos.
 - Post: Se retorna la distancia euclidiana entre los dos vértices.
- operator(otro): b, compara el vértice analizado con otro y retorna si son iguales.
 - Pre: Ambos vértices deben estar definidos.
 - Post: Se retorna verdadero si las coordenadas de ambos vértices son iguales, de lo contrario, falso.

Cara

El TAD cara es esencial para la representación de superficies en el espacio tridimensional. Una cara está compuesta por un conjunto de vértices que al ser conectados forman una superficie plana dentro de un objeto. Este TAD permite definir, manipular y analizar las caras de un objeto, proporcionando métodos para gestionar los vértices que la componen y calcular propiedades geométricas como el área.

Estado

- Vértices: (vertices), plantilla, contiene los vértices que definen la cara en el espacio tridimensional.
 - El contenedor debe tener al menos tres vértices para formar una cara válida.

Interfaz

- Cara(): cara, crea una cara por defecto sin vertices.
- Cara(vertices): cara, crea una cara utilizando un conjunto dado de vértices.
 - Pre: vertices debe contener al menos tres elementos.
- getVertices(): vertices, retorna los vértices que componen la cara.
 - Pre: La cara debe tener vértices definidos.
- getNumeroVertices(): numero, retorna el número de vértices que componen la cara.
 - Pre: La cara debe estar definida.
- setVertices(vertices), establece un nuevo conjunto de vértices para la cara.
 - Pre: vertices debe tener al menos tres vértices.
- calcularArea(): area, calcula y retorna el área de la cara.
 - Pre: La cara debe tener vértices suficientes y válidos para formar una superficie.
- agregarVertice(vertice), añade un nuevo vértice a la lista de vértices de la cara.
 - Pre: vertice debe ser un vértice válido.

Objeto

El TAD objeto representa una entidad tridimensional completa que está compuesta por un conjunto de caras, cada una de las cuales está definida por sus vértices. Este TAD es esencial para modelar y manipular objetos completos en el espacio, permitiendo realizar operaciones como agregar caras, calcular la caja envolvente del objeto, y obtener representaciones textuales de su estructura. El objeto también tiene un nombre que lo identifica de manera única dentro del sistema.

Estado

- Nombre_del_objeto: (nombre), cadena de caracteres, identifica al objeto dentro del sistema.
 - nombre debe ser una cadena de caracteres válida, no puede haber otro objeto con ese nombre.
- Caras: (caras), plantilla, contiene las caras que componen el objeto tridimensional.

Interfaz

- Objeto(): objeto, crea un objeto por defecto sin nombre y sin caras.
- Objeto(nombre): objeto, crea un objeto con un nombre especificado, pero sin caras.
 - Pre: nombre debe ser una cadena válida, no puede haber otro objeto con ese mismo nombre.
- Objeto (nombre, caras): objeto, crea un objeto con un nombre y un conjunto de caras especificados.
 - Pre: nombre debe ser una cadena válida, no puede haber otro objeto con ese mismo nombre.
- getNombre(): nombre, retorna el nombre del objeto.
 - Pre: El objeto debe tener un nombre definido.
- getCaras(): caras, retorna las caras que componen el objeto.
 - Pre: El objeto debe tener caras definidas.
- setNombre(nombre), establece un nuevo nombre para el objeto.
 - Pre: nombre debe ser una cadena válida; ningún otro objeto puede tener el mismo nombre.
- setCaras(caras), establece un nuevo conjunto de caras para el objeto.
- agregarCara(cara), añade una nueva cara a la lista de caras del objeto.
 - Pre: cara debe ser una cara válida.
- calcularCajaEnvolvente(xmin, ymin, zmin, xmax, ymax, zmax), calcula los límites de la caja envolvente que contiene al objeto.
 - Pre: El objeto debe tener vértices y caras definidas.
 - Post: Se calculan los límites mínimos y máximos en los ejes X, Y y Z que encierran el objeto.
- toString(): cadena de caracteres, retorna una representación textual del objeto, incluyendo su nombre y detalles de sus caras.

Menú

El TAD menu gestiona las operaciones relacionadas con la carga, manipulación, y almacenamiento de objetos tridimensionales, así como la interacción con los usuarios a través de un menú. El Menu controla y organiza los objetos cargados en memoria, permitiendo ejecutar una serie de operaciones que facilitan la construcción y análisis de modelos 3D.

Estado

- Objetos_cargados: (objetosCargados), lista, almacena los objetos tridimensionales que han sido cargados en memoria.

Interfaz

- Menu(): Menu, inicializa un menú vacío con la capacidad de gestionar objetos.
- objetoCargado(nombre_archivo): b, verifica si un objeto con un nombre dado ya ha sido cargado en memoria.
 - Post: Retorna verdadero si el objeto ya está cargado, falso en caso contrario.
- mostrarMenu(), muestra las opciones del menú al usuario.
- manejarEntradaUsuario(), gestiona y procesa la entrada del usuario desde el menú.
- mostrarAyuda(), muestra una menu de ayuda sobre cómo utilizar el sistema.
- salirPrograma(), cierra el programa.
- cargarObjeto(nombre_archivo), carga un objeto desde un archivo y lo almacena en memoria.
 - Pre: nombre_archivo debe ser una cadena válida que apunte a un archivo existente que no haya sido cargado previamente.
- listarObjetosCargados(), muestra una lista de todos los objetos que están actualmente cargados en memoria.
- calcularCajaEnvolvente(nombre_objeto), calcula y muestra la caja envolvente de un objeto específico.
 - Pre: nombre_objeto debe referirse a un objeto cargado en memoria.
- crearCajaDeAjusteGlobal(), calcula y muestra la caja envolvente que incluye todos los objetos cargados en memoria.
 - Pre: Deben existir objetos cargados en memoria.
- descargarObjeto(nombre_objeto), elimina un objeto específico de la memoria.
 - Pre: nombre_objeto debe referirse a un objeto cargado en memoria.
- guardarObjetoEnArchivo(nombre_objeto, nombre_archivo), guarda un objeto cargado en memoria en un archivo.
 - Pre: nombre_objeto debe referirse a un objeto cargado en memoria; nombre_archivo debe ser una cadena válida.
- verticeCercanoObjeto(px, py, pz, nombre_objeto), encuentra y muestra el vértice más cercano a un punto específico dentro de un objeto.
 - Pre: px, py, pz deben ser valores numéricos que representan el punto de interés; nombre_objeto debe referirse a un objeto cargado en memoria.
- verticeCercanoTodosObjetos(px, py, pz), encuentra y muestra el vértice más cercano a un punto específico considerando todos los objetos cargados en memoria.
 - Pre: px, py, pz deben ser valores numéricos que representan el punto de interés.
- verticesCercanosCaja(nombre_objeto), muestra todos los vértices cercanos a la caja envolvente de un objeto específico.
 - Pre: nombre_objeto debe referirse a un objeto cargado en memoria.
- rutaCortaVertices(v1, v2, nombre_objeto), calcula y muestra la ruta más corta entre dos vértices dentro de un objeto específico.

- Pre: v1 y v2 deben ser índices válidos de vértices dentro de nombre_objeto.
- rutaCortaCentro(v1, nombre_objeto), calcula y muestra la ruta más corta entre un vértice y el centro del objeto.
 - Pre: v1 debe ser un índice válido de un vértice dentro de nombre_objeto.

Diagrama de Relación de TADs:

El diagrama presentado en la figura 9 detalla las relaciones entre los TADs utilizados en el sistema: Vértice, Cara, Objeto, y Menú. Estos TADs permiten la manipulación y almacenamiento de modelos tridimensionales. El vértice actúa como la unidad básica de la geometría, representando un punto en el espacio tridimensional mediante coordenadas (x, y, z). Su importancia radica en que es un componente fundamental de la cara, que es una superficie plana definida por un conjunto de vértices. Esta relación entre vértice y cara se muestra claramente en el diagrama mediante una conexión directa, lo que indica que una cara está formada por múltiples vértices. A su vez, el Objeto es una entidad más compleja que encapsula un modelo tridimensional completo compuesto por varias caras. La relación entre objeto y cara, también representada en el diagrama, muestra que un objeto se construye a partir de estas, definiendo su estructura en el espacio tridimensional.

El menú, por otra parte, se representa como el controlador principal del sistema, encargado de gestionar la interacción del usuario con los objetos tridimensionales. El menú administra los objetos cargados, permitiendo al usuario realizar operaciones como cargar, listar, manipular y guardar objetos, entre otras. En el diagrama, la conexión entre menú y objeto refleja esta relación de control, mostrando que el menú maneja un contenedor de objetos y facilita su manipulación dentro del sistema.

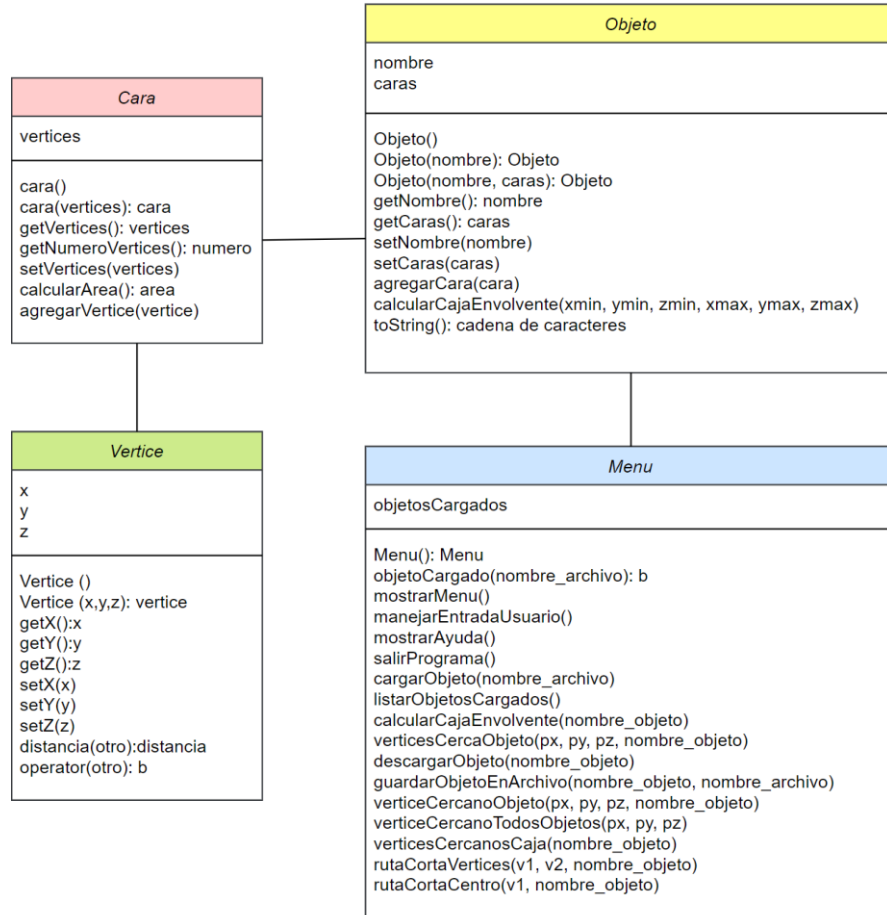


Figura 9. Diagrama de relación de TADs

2.3 Plan de Pruebas

Inicialmente, se establecerá que no se realizarán pruebas respecto a mensajes de éxito y fracaso, ya que estos fueron abordados en una entrega anterior. Para el comando envolvente, se llevará a cabo un plan de pruebas que garantizará la funcionalidad del componente tanto antes como después de la codificación. Esto incluirá la verificación de casos con memoria vacía y con objetos cargados en memoria, con un enfoque en evaluar la capacidad del sistema para manejar diferentes escenarios y garantizar la integridad de los datos.

En primer lugar, se prepararán pruebas previas a la codificación para asegurarse de que el comando envolvente pueda manejar adecuadamente las cajas envolventes. Esto

implicará verificar que el comando funcione correctamente en el contexto de diferentes octantes tridimensionales. Esto incluye probar el sistema con objetos en los ocho octantes del plano tridimensional y asegurarse de que los vértices estén correctamente manejados en cada caso. Es importante señalar que los octantes serán considerados de la siguiente manera:

1. Primer octante: $(x > 0), (y > 0), (z > 0)$
2. Segundo octante: $(x < 0), (y > 0), (z > 0)$
3. Tercer octante: $(x < 0), (y < 0), (z > 0)$
4. Cuarto octante: $(x > 0), (y < 0), (z > 0)$
5. Quinto octante: $(x > 0), (y > 0), (z < 0)$
6. Sexto octante: $(x < 0), (y > 0), (z < 0)$
7. Séptimo octante: $(x < 0), (y < 0), (z < 0)$
8. Octavo octante: $(x > 0), (y < 0), (z < 0)$

Adicionalmente, se utilizarán valores extremos y se validarán los archivos para asegurar que los vértices de las cajas envolventes se inicialicen adecuadamente con la librería `<limits>`, prestando especial atención al manejo de decimales y valores negativos.

Una vez que se haya completado la codificación, se llevarán a cabo pruebas específicas para verificar su funcionalidad. Las pruebas se centrarán en dos escenarios principales: uno en el que no hay objetos en memoria y otro en el que hay múltiples objetos en memoria, este último constando de varias pruebas. El objetivo es confirmar que el comando maneja correctamente la creación de la caja envolvente global, tanto cuando la memoria está vacía como cuando contiene objetos con diferentes características. Si se encuentran errores durante estas pruebas, se realizarán ajustes en el código y se crearán nuevas pruebas específicas para abordar cualquier problema identificado.

Planificación de Pruebas

Para el comando envolvente, se realizarán las siguientes pruebas:

1. Prueba con memoria vacía: Se verificará que el sistema muestre el mensaje adecuado cuando no hay objetos en memoria. El sistema debe indicar que la memoria está vacía, lo que confirma que el comando maneja correctamente la ausencia de objetos.
2. Pruebas con objetos en memoria: Por cada prueba, se cargará un conjunto de objetos en memoria, cada uno con particularidades que pongan a prueba casos límites, posibles omisiones o errores de implementación. Luego se ejecutará el comando envolvente para calcular la caja envolvente global que incluya todos los objetos en memoria. Las pruebas confirmarán que la caja envolvente global se calcula correctamente, que incluye todos los vértices de los objetos y que se añade a la memoria con el nombre `env_global`. Se validará que los puntos extremos (`pmin` y `pmax`) de la caja envolvente sean precisos y reflejen

correctamente los límites globales de todos los objetos, realizando además una representación en el plano cartesiano.

- A.** Prueba en un solo octante: En esta prueba, se colocarán varios objetos en un solo octante del espacio cartesiano (por ejemplo, el primer octante donde todas las coordenadas son positivas). El objetivo es verificar que la caja envolvente global se calcule correctamente dentro de este espacio restringido y que incluya todos los vértices de los objetos. Se confirmará que los valores de p_{min} y p_{max} de la caja envolvente reflejan con precisión los límites del octante en el que se encuentran los objetos.
- B.** Prueba en todos los octantes: Esta prueba implica la creación y carga de objetos distribuidos en los ocho octantes del espacio tridimensional, lo que significa que las coordenadas de los vértices tendrán combinaciones positivas y negativas. Se verificará que el comando envolvente calcule correctamente la caja envolvente global que abarque todos los objetos, sin importar en qué octante se encuentren. La prueba confirmará que los límites de la caja envolvente global (p_{min} y p_{max}) cubren adecuadamente todo el espacio ocupado por los objetos en todos los octantes.
- C.** Prueba con decimales: En esta prueba, los objetos cargados tendrán vértices con coordenadas decimales. El propósito es verificar que el comando envolvente maneje correctamente los valores decimales y calcule una caja envolvente precisa que incluya todos los vértices. Se confirmará que la precisión de los puntos extremos (p_{min} y p_{max}) no se ve afectada por el uso de números decimales.
- D.** Prueba con malla de un objeto: Se cargará un solo objeto con una malla compleja, posiblemente con muchas caras y vértices, para evaluar si el comando envolvente es capaz de calcular la caja envolvente global de manera eficiente y precisa. Esta prueba validará que el sistema puede manejar un objeto detallado sin errores, calculando correctamente los límites de la caja envolvente.
- E.** Prueba con malla global: Prueba con malla global: En esta prueba a partir de una malla global se calculará su envolvente que como resultado deberá dar la misma malla global.
- F.** Prueba con vértices máximos: Esta prueba se centrará en la capacidad del sistema para manejar objetos con coordenadas de vértices que alcanzan valores máximos dentro de los límites del sistema (por ejemplo, valores cercanos al máximo permitido por el tipo de dato). El objetivo es asegurar que el comando envolvente puede calcular la

caja envolvente global sin desbordamientos o errores numéricos, y que los límites calculados (p_{min} y p_{max}) son correctos.

- G. Prueba con vértices mínimos: Similar a la prueba F, pero en este caso, los objetos tendrán vértices con coordenadas que alcanzan valores mínimos dentro de los límites del sistema (por ejemplo, valores muy cercanos al mínimo permitido por el tipo de dato). Se verificará que el comando envolvente maneje correctamente estos valores extremos, calculando una caja envolvente global precisa que incluya todos los objetos, y que los puntos extremos (p_{min} y p_{max}) sean los correctos.

Ejecución de Pruebas

Prueba con memoria vacía

Para llevar a cabo la prueba con memoria vacía, se siguió una serie de pasos diseñados para verificar que el sistema maneje adecuadamente la ausencia de objetos en memoria y muestre el mensaje correspondiente. Primero, se inició el sistema asegurándose de que no existieran objetos cargados en memoria. Esto se logró reiniciando el sistema, pero también se puede realizar mediante el comando descargar. Con la memoria vacía, se procedió a ejecutar el comando envolvente que debería intentar calcular la caja envolvente global de todos los objetos presentes en memoria.

El resultado obtenido en esta prueba fue el esperado, se observó en la consola el mensaje “(Memoria vacía) Ningún objeto ha sido cargado en memoria” como puede ser visualizado en la figura 10. Se usó el comando listado para verificar que efectivamente no existía nada en la memoria del sistema.

```
~/Estructuras$ g++ -o a *.cpp
~/Estructuras$ ./a
Bienvenido al sistema de gestión de objetos. Digite 'ayuda' en caso de requerirlo.
$ envolvente
(Memoria vacía) Ningún objeto ha sido cargado en memoria.
El comando 'envolvente' se ejecutó en 1.442e-05 segundos.
$
```

Figura 10. Ejecución prueba con memoria vacía

Prueba en un solo octante

Para llevar a cabo la prueba de un solo octante, se prepararon tres cubos ubicados completamente dentro del primer octante del espacio cartesiano, con todas sus coordenadas positivas. Los cubos fueron definidos en archivos de texto utilizando el formato

especificado en el proyecto. Se cargaron en memoria utilizando el comando cargar, asegurando que cada objeto se incorporara correctamente en el sistema sin errores. Con los tres cubos cargados, se procedió a ejecutar el comando envolvente para calcular la caja envolvente global que abarcara todos los vértices de los objetos cargados en memoria como se puede visualizar en la figura 11.

```
jaz@jaz-VirtualBox: /Estructuras-de-datos-main$ g++ -o programa *.cpp
jaz@jaz-VirtualBox: /Estructuras-de-datos-main$ ./programa
Bienvenido al sistema de gestión de objetos. Digite 'ayuda' en caso de requerirlo.
$ cargar cubo1.txt
(Resultado exitoso) El objeto Cubo_1 ha sido cargado exitosamente desde el archivo cubo1.txt.
$ cargar cubo2.txt
(Resultado exitoso) El objeto Cubo_2 ha sido cargado exitosamente desde el archivo cubo2.txt.
$ cargar cubo3.txt
(Resultado exitoso) El objeto Cubo_3 ha sido cargado exitosamente desde el archivo cubo3.txt.
$ envolvente
(Resultado exitoso) La caja de ajuste global se ha generado con el nombre caja_global y se ha agregado a los objetos en memoria.
$ envolvente
(Resultado exitoso) La caja de ajuste global se ha generado con el nombre caja_global y se ha agregado a los objetos en memoria.
$ guardar caja_global global.txt
(Resultado exitoso) La información del objeto caja_global ha sido guardada exitosamente en el archivo global.txt.
```

Figura 11. Ejecución del comando envolvente

El resultado esperado de la caja envolvente global era una caja definida por los puntos $p_{min} = (1, 1, 1)$ y $p_{max} = (6, 6, 6)$, los cuales representan los límites extremos de los vértices de los tres cubos en el espacio tridimensional. Tras la ejecución del comando envolvente, se verificó que el sistema generó correctamente una caja envolvente global con el nombre env_global, y que las coordenadas de los puntos extremos coincidían con las esperadas. Además, el sistema mostró un mensaje de éxito, indicando que la operación se había completado sin problemas. Para mayor detalle visualizar las imágenes 12 y 13.

cubo1.txt	cubo2.txt	cubo3.txt	global.txt
Cubo_1	Cubo_2	Cubo_3	caja_global
8	8	8	8
1 1 1	3 3 3	5 5 5	1 1 1
2 1 1	4 3 3	6 5 5	1 1 6
2 2 1	4 4 3	6 6 5	1 6 6
1 2 1	3 4 3	5 6 5	1 6 1
1 1 2	3 3 4	5 5 6	6 1 1
2 1 2	4 3 4	6 5 6	6 1 6
2 2 2	4 4 4	6 6 6	6 6 6
1 2 2	3 4 4	5 6 6	6 6 1
4 0 1 2 3	4 0 1 2 3	4 0 1 2 3	4 0 1 2 3
4 4 5 6 7	4 4 5 6 7	4 4 5 6 7	4 4 5 6 7
4 0 1 5 4	4 0 1 5 4	4 0 1 5 4	4 0 1 5 4
4 1 2 6 5	4 1 2 6 5	4 1 2 6 5	4 1 2 6 5
4 2 3 7 6	4 2 3 7 6	4 2 3 7 6	4 2 3 7 6
4 3 0 4 7	4 3 0 4 7	4 3 0 4 7	4 3 0 4 7
-1	-1	-1	-1

Figura 12. Archivos de texto de cubos en primer octante y caja envolvente global

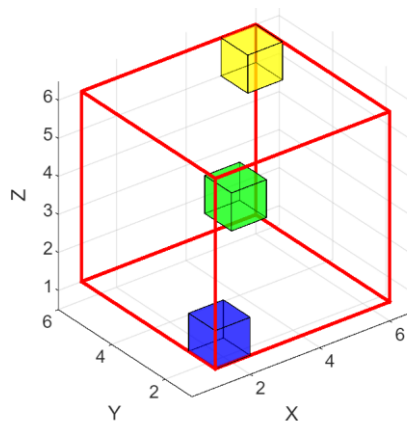


Figura 13. Cubos en primer octante con caja envolvente global

Prueba en todos los octantes

En primer lugar, se definieron y cargaron en memoria ocho cubos, cada uno ubicado en un octante diferente del espacio tridimensional. Las coordenadas de los vértices de cada cubo se aseguraron de cubrir desde el origen $(0, 0, 0)$ hasta una unidad en cada dirección, positiva o negativa según el octante correspondiente. Los cubos se cargaron utilizando el comando cargar, y se confirmó que todos los objetos se habían almacenado correctamente en la memoria del sistema.

Una vez cargados todos los cubos, se ejecutó el comando envolvente para calcular la caja envolvente global que abarque todos los vértices de los objetos cargados. El sistema realizó este cálculo de manera eficiente, determinando que los puntos extremos de la caja envolvente global correspondían a las coordenadas mínimas y máximas en los ejes x , y , y z . Los valores calculados para p_{min} y p_{max} fueron $(-1, -1, -1)$ y $(1, 1, 1)$, respectivamente, lo que concuerda con los límites esperados. El sistema mostró un mensaje de éxito, indicando que la caja envolvente global había sido generada correctamente y se había agregado a los objetos en memoria con el nombre `env_global`. En las imágenes 13 y 14 pueden ser visualizados los resultados de la prueba a profundidad.

global.txt
caja_global
8
-1 -1 -1
-1 -1 1
-1 1 1
-1 1 -1
1 -1 -1
1 -1 1
1 1 1
1 1 -1
4 0 1 2 3
4 4 5 6 7
4 0 1 5 4
4 1 2 6 5
4 2 3 7 6
4 3 0 4 7
-1

Figura 14. Archivo de caja envolvente global de cubos en cada octante

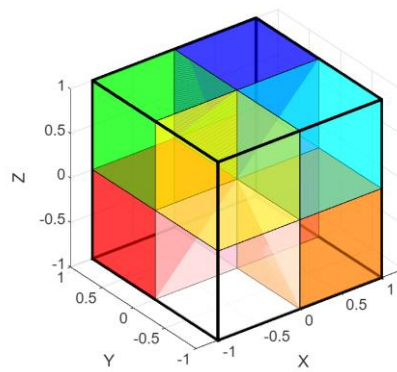


Figura 15. Cubos en cada octante con caja envolvente global

```

~/Estructuras$ g++ -o a *.cpp
~/Estructuras$ ./a
Bienvenido al sistema de gestión de objetos. Digite 'ayuda' en caso de requerirlo.
$ cargar cubo1.txt
(Resultado exitoso) El objeto Cubo_1 ha sido cargado exitosamente desde el archivo cubo1.txt.
El comando 'cargar' se ejecutó en 0.00013517 segundos.
$ cargar cubo2.txt
(Resultado exitoso) El objeto Cubo_2 ha sido cargado exitosamente desde el archivo cubo2.txt.
El comando 'cargar' se ejecutó en 0.00012373 segundos.
$ cargar cubo3.txt
(Resultado exitoso) El objeto Cubo_3 ha sido cargado exitosamente desde el archivo cubo3.txt.
El comando 'cargar' se ejecutó en 0.00012082 segundos.
$ cargar cubo4.txt
(Resultado exitoso) El objeto Cubo_4 ha sido cargado exitosamente desde el archivo cubo4.txt.
El comando 'cargar' se ejecutó en 0.000296409 segundos.
$ cargar cubo5.txt
(Resultado exitoso) El objeto Cubo_5 ha sido cargado exitosamente desde el archivo cubo5.txt.
El comando 'cargar' se ejecutó en 7.159e-05 segundos.
$ cargar cubo6.txt
(Resultado exitoso) El objeto Cubo_6 ha sido cargado exitosamente desde el archivo cubo6.txt.
El comando 'cargar' se ejecutó en 0.00022605 segundos.
$ cargar cubo7.txt
(Resultado exitoso) El objeto Cubo_7 ha sido cargado exitosamente desde el archivo cubo7.txt.
El comando 'cargar' se ejecutó en 8.018e-05 segundos.
$ cargar cubo8.txt
(Resultado exitoso) El objeto Cubo_8 ha sido cargado exitosamente desde el archivo cubo8.txt.
El comando 'cargar' se ejecutó en 0.00024246 segundos.
$ envolverte
(Resultado exitoso) La caja de ajuste global se ha generado con el nombre caja_global y se ha
agregado a los objetos en memoria.
El comando 'envolverte' se ejecutó en 3.754e-05 segundos.
$ guardar caja_global octetos.txt
(Resultado exitoso) La información del objeto caja_global ha sido guardada exitosamente en el
archivo octetos.txt.
El comando 'guardar' se ejecutó en 0.00054673 segundos.
$ salir
Saliendo del programa...

```

Figura 16. Ejecución prueba en primer octante con caja envolvente global

Prueba con decimales

Para la prueba con decimales, se utilizaron las coordenadas de una pirámide con el objetivo de verificar que el sistema pueda manejar adecuadamente objetos con coordenadas decimales y calcular de manera precisa la caja envolvente que los contiene. Inicialmente, se definieron y cargaron en memoria los vértices de la pirámide, los cuales formaron una estructura tridimensional con una base cuadrada y un vértice superior (ápice). Estas coordenadas aseguraron que la pirámide se representara correctamente en el sistema.

Posteriormente, se ejecutó el comando `envolverte` para calcular la caja envolvente de tal forma que abarcara todos los vértices de la pirámide. El sistema determinó con precisión los puntos extremos de la caja envolvente, ubicando `pmin` y `pmax` en las coordenadas mínimas y máximas correspondientes en cada eje. Los valores esperados para estos puntos eran `pmin` en (0, 0, 0) y `pmax` en (2.5, 2.5, 2.5). Los resultados obtenidos confirmaron estas expectativas como puede ser observado en las figuras 17 y 18.

Piramide.txt	env.txt
Piramide	caja_global
5	8
0 0 0	0 0 0
2.5 0 0	0 0 2.5
2.5 2.5 0	0 2.5 2.5
0 2.5 0	0 2.5 0
1.25 1.25 2.5	2.5 0 0
4 0 1 2 3	2.5 0 2.5
3 0 1 4	2.5 2.5 2.5
3 1 2 4	2.5 2.5 0
3 2 3 4	4 0 1 2 3
3 3 0 4	4 4 5 6 7
-1	4 0 1 5 4
	4 1 2 6 5
	4 2 3 7 6
	4 3 0 4 7
	-1

Figura 17. Archivos de texto prueba con decimales

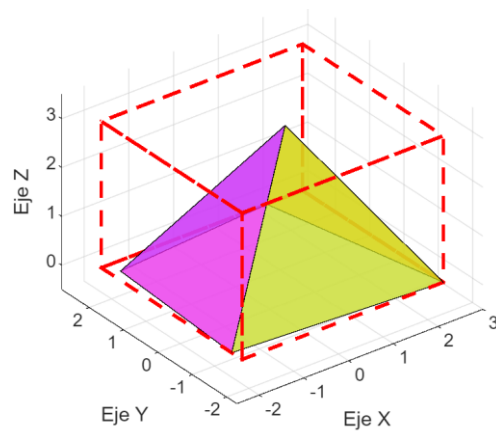


Figura 18. Pirámide y caja envolvente

```
~/Estructuras$ g++ -o programa *.cpp
~/Estructuras$ ./programa
Bienvenido al sistema de gestión de objetos. Digite 'ayuda' en caso de requerirlo.
$ cargar Piramide.txt
(Resultado exitoso) El objeto Piramide ha sido cargado exitosamente desde el archivo Piramide.txt.
El comando 'cargar' se ejecutó en 8.341e-05 segundos.
$ envolvente
(Resultado exitoso) La caja de ajuste global se ha generado con el nombre caja_global y se ha agregado a los objetos en memoria.
El comando 'envolvente' se ejecutó en 3.136e-05 segundos.
$ guardar caja_global env1.txt
(Resultado exitoso) La información del objeto caja_global ha sido guardada exitosamente en el archivo env1.txt.
El comando 'guardar' se ejecutó en 0.00055095 segundos.
$
```

Figura 19. Consola y tiempos de ejecución en prueba con decimales

Prueba con una malla de objeto

Para la ejecución de la prueba con una malla compleja, se cargó un objeto denominado Mesh_HexagonalPrism que contiene una estructura de un prisma hexagonal. Este objeto se compone de doce vértices y múltiples caras, lo que lo convierte en una malla detallada y más compleja que las pruebas anteriores. El propósito de esta prueba fue evaluar si el comando envolvente es capaz de calcular la caja envolvente global de manera eficiente y precisa, incluso cuando se enfrenta a un objeto con una cantidad considerable de vértices y caras.

Una vez que se cargó la malla en el sistema, se ejecutó el comando envolvente para determinar la caja envolvente global que abarca todos los vértices de la malla hexagonal. El sistema debía calcular con precisión los puntos extremos de la caja envolvente, identificando pmin y pmax en las coordenadas mínimas y máximas de los ejes X, Y y Z. Los valores esperados para estos puntos eran pmin en (-0.866, -1.000, -1.000) y pmax en (0.866, 1.000, 1.000), reflejando correctamente los límites extremos de la malla. La prueba fue exitosa, ya que el sistema logró manejar el objeto detallado sin errores, calculando correctamente los límites de la caja envolvente como puede ser visualizado en las figuras 20 y 21.

Prism.txt	env.txt
Mesh_HexagonalPrism	caja_global
12	8
0.000 1.000 1.000	-0.866 -1 -1
0.866 0.500 1.000	-0.866 -1 1
0.866 -0.500 1.000	-0.866 1 1
0.000 -1.000 1.000	-0.866 1 -1
-0.866 -0.500 1.000	0.866 -1 -1
-0.866 0.500 1.000	0.866 -1 1
0.000 1.000 -1.000	0.866 1 1
0.866 0.500 -1.000	0.866 1 -1
0.866 -0.500 -1.000	4 0 1 2 3
0.000 -1.000 -1.000	4 4 5 6 7
-0.866 -0.500 -1.000	4 0 1 5 4
-0.866 0.500 -1.000	4 1 2 6 5
3 0 1 2	4 2 3 7 6
3 0 2 3	4 3 0 4 7
3 0 3 4	-1
3 0 4 5	
3 6 7 8	
3 6 8 9	
3 6 9 10	
3 6 10 11	
4 0 1 7 6	
4 1 2 8 7	
4 2 3 9 8	
4 3 4 10 9	
4 4 5 11 10	
4 5 0 6 11	
-1	

Figura 20. Archivos de texto prueba con una malla de objeto

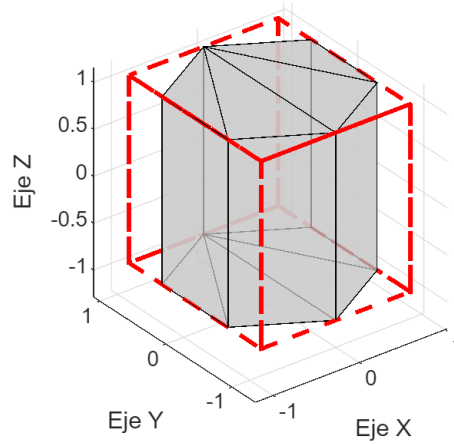


Figura 21. Prisma hexagonal y su caja envolvente

```

$ ./test_objetos.py prog_000
Bienvenido al sistema de gestión de objetos. Digite 'ayuda' en caso de requerirlo.
$ cargar jaz.txt
(Resultado exitoso) El objeto Mesh_HexagonalPrism ha sido cargado exitosamente desde el archivo jaz.txt.
El comando 'cargar' se ejecutó en 0.00020271 segundos.
$ envolvente
(Resultado exitoso) La caja de ajuste global se ha generado con el nombre caja_global y se ha agregado a los objetos
en memoria.
El comando 'envolvente' se ejecutó en 0.00011832 segundos.
$ guardar caja_global envjaz.txt
(Resultado exitoso) La información del objeto caja_global ha sido guardada exitosamente en el archivo envjaz.txt.
El comando 'guardar' se ejecutó en 0.00074637 segundos.
$

```

Figura 22. Ejecución prueba con una malla de objeto

Prueba con malla global

Para la ejecución de la prueba con malla global, se tomó como referencia la caja envolvente global resultante de la prueba con todos los octantes (Prueba B). Esta caja envolvente, que ya había sido calculada anteriormente para abarcar completamente los ocho cubos distribuidos en los diferentes octantes, fue utilizada como la malla de entrada en esta prueba. El objetivo era verificar si el sistema es capaz de calcular una nueva caja envolvente para esta malla global y si, efectivamente, dicha envolvente resulta ser la misma malla global utilizada como entrada.

Una vez cargada la malla global en el sistema, se ejecutó el comando envolvente para calcular nuevamente la caja envolvente. El resultado esperado era que la nueva caja envolvente coincidiera exactamente en tamaño y características con la malla global original. La prueba fue exitosa, ya que el sistema calculó la caja envolvente de manera precisa, y ambas quedaron del mismo tamaño y con las mismas características, validando así el correcto funcionamiento del sistema. Este resultado se observa claramente en la Figura 23.

global.txt	env_global.txt
caja_global	caja_global
8	8
-1 -1 -1	-1 -1 -1
-1 -1 1	-1 -1 1
-1 1 1	-1 1 1
-1 1 -1	-1 1 -1
1 -1 -1	1 -1 -1
1 -1 1	1 -1 1
1 1 1	1 1 1
1 1 -1	1 1 -1
4 0 1 2 3	4 0 1 2 3
4 4 5 6 7	4 4 5 6 7
4 0 1 5 4	4 0 1 5 4
4 1 2 6 5	4 1 2 6 5
4 2 3 7 6	4 2 3 7 6
4 3 0 4 7	4 3 0 4 7
-1	-1

Figura 23. Archivos prueba con una malla global

```
~/Estructuras$ ./programa
Bienvenido al sistema de gestión de objetos. Digite 'ayuda' en caso de requerirlo.
$ cargar octetos.txt
(Resultado exitoso) El objeto caja_global ha sido cargado exitosamente desde el archivo octetos.txt.
El comando 'cargar' se ejecutó en 0.000102579 segundos.
$ envolverte
(Resultado exitoso) La caja de ajuste global se ha generado con el nombre caja_global y se ha agregado a los objetos
en memoria.
El comando 'envolverte' se ejecutó en 4.75e-05 segundos.
$ guardar caja_global caja_global_global.txt
(Resultado exitoso) La información del objeto caja_global ha sido guardada exitosamente en el archivo caja_global_gl
obal.txt.
El comando 'guardar' se ejecutó en 0.00052357 segundos.
$
```

Figura 24. Ejecución prueba con una malla global

objeto en el primer octante:

En este caso, se cargó un objeto cuyas coordenadas de todos los vértices se encuentran en el primer octante, es decir, todas las coordenadas x, y, y z son positivas. Posteriormente, se realizó la prueba para verificar si el método calcularCajaEnvolvente determina correctamente los puntos extremos de la caja envolvente. El resultado esperado es que los valores máximos y mínimos para cada eje se calculen sin ningún inconveniente. El archivo de texto que corresponde al objeto que se cargó es el siguiente:

```

1  Cubo_PrimerOctante
2  8
3  0 0 0
4  10 0 0
5  10 10 0
6  0 10 0
7  0 0 10
8  10 0 10
9  10 10 10
10 0 10 10
11 4 0 1 2 3
12 4 4 5 6 7
13 4 0 1 5 4
14 4 1 2 6 5
15 4 2 3 7 6
16 4 3 0 4 7
17 -1

```

Figura 25. Contenido de cuboprimer.txt

Valores esperados:

- Mínimos (xmin, ymin, zmin): 0, 0, 0
- Máximos (xmax, ymax, zmax): 10, 10, 10

Pasos por seguir:

1. Se inicia el proceso cargando el archivo de texto correspondiente al objeto con el comando cargar cuboprimer.txt.
2. A continuación, se genera la caja envolvente del cubo mediante el comando envolvente Cubo_PrimerOctante.
3. Finalmente, el resultado se guarda en un nuevo archivo de texto utilizando el comando guardar env_Cubo_PrimerOctante envolventeCubo1.txt.

```

~/Estructuras$ g++ -std=c++11 -o programa *.cpp
~/Estructuras$ ./programa
Bienvenido al sistema de gestión de objetos. Digite 'ayuda' en caso de requerirlo.
$ cargar cuboprimer.txt
(Resultado exitoso) El objeto Cubo_PrimerOctante ha sido cargado exitosamente desde el archivo cuboprimer.txt.
El comando 'cargar' se ejecutó en 8.644e-05 segundos.
$ envolvente Cubo_PrimerOctante
(Resultado exitoso) La caja envolvente del objeto Cubo_PrimerOctante se ha generado con el nombre env_Cubo_PrimerOctante y se ha agregado a los objetos en memoria.
El comando 'envolvente' se ejecutó en 3.602e-05 segundos.
$ guardar env_Cubo_PrimerOctante envolventeCubo1
(Resultado exitoso) La información del objeto env_Cubo_PrimerOctante ha sido guardada exitosamente en el archivo envolventeCubo1.
El comando 'guardar' se ejecutó en 0.00075671 segundos.

```

Figura 26. Ejecución de los comandos correspondientes

```

envolventeCubo1
1  env_Cubo_PrimerOctante
2  8
3  0 0 0
4  0 0 10
5  0 10 10
6  0 10 0
7  10 0 0
8  10 0 10
9  10 10 10
10 10 10 0
11 4 0 1 2 3
12 4 4 5 6 7
13 4 0 1 5 4
14 4 1 2 6 5
15 4 2 3 7 6
16 4 3 0 4 7
17 -1
18

```

Figura 27. Contenido de la envoltente creada

En esta imagen, se puede observar que los valores esperados si corresponden a los generados por el programa, donde la coordenada mínima es 0,0,0 y la máxima es 10,10,10

cubo en el tercer octante

Este caso analiza un cubo cuyas coordenadas de vértices están en el tercer octante, lo que implica que todas las coordenadas x, y, y z son negativas. Este escenario es útil para verificar que el algoritmo maneja correctamente valores negativos.

El archivo de texto que corresponde al objeto que se cargó es el siguiente:

```

cubotercer.txt
1  Cubo_TercerOctante
2  8
3  -10 -10 -10
4  -20 -10 -10
5  -20 -20 -10
6  -10 -20 -10
7  -10 -10 -20
8  -20 -10 -20
9  -20 -20 -20
10 -10 -20 -20
11 4 0 1 2 3
12 4 4 5 6 7
13 4 0 1 5 4
14 4 1 2 6 5
15 4 2 3 7 6
16 4 3 0 4 7
17 -1
18
19

```

Figura 28. Contenido del objeto a cargar (cubotercer.txt)

Valores esperados:

- Mínimos (xmin, ymin, zmin): -20, -20, -20
- Máximos (xmax, ymax, zmax): -10, -10, -10

Pasos por seguir:

1. Primero, se carga el archivo de texto Cubo_TercerOctante.txt que contiene la información del objeto.
2. Luego, se procede a generar la caja envolvente utilizando el comando envolver Cubo_TercerOctante.
3. Finalmente, el resultado se guarda en un archivo de texto con el comando guardar env_Cubo_TercerOctante resultado.txt.

```
$ cargar cubotercer.txt
(Resultado exitoso) El objeto Cubo_TercerOctante ha sido
cargado exitosamente desde el archivo cubotercer.txt.
El comando 'cargar' se ejecutó en 8.423e-05 segundos.
$ envolver Cubo_TercerOctante
(Resultado exitoso) La caja envolvente del objeto Cubo_T
ercerOctante se ha generado con el nombre env_Cubo_Terce
rOctante y se ha agregado a los objetos en memoria.
El comando 'envolver' se ejecutó en 2.4149e-05 segundos.
$ guardar env_Cubo_TercerOctante envolverCubo3
(Resultado exitoso) La información del objeto env_Cubo_T
ercerOctante ha sido guardada exitosamente en el archivo
envolverCubo3.
El comando 'guardar' se ejecutó en 0.00052352 segundos.
```

Figura 29. Ejecución de los comandos correspondientes

```
envolverCubo3
1  env_Cubo_TercerOctante
2  8
3  -20 -20 -20
4  -20 -20 -10
5  -20 -10 -10
6  -20 -10 -20
7  -10 -20 -20
8  -10 -20 -10
9  -10 -10 -10
10 -10 -10 -20
11 4 0 1 2 3
12 4 4 5 6 7
13 4 0 1 5 4
14 4 1 2 6 5
15 4 2 3 7 6
16 4 3 0 4 7
17 -1
18
```

Figura 30. Contenido de la envolvente creada

En esta imagen, se puede observar que los valores esperados si corresponden a los generados por el programa, donde la coordenada mínima es -20,-20,-20 y la máxima es -10,-10,-10

En este caso, el cubo está ubicado en el origen del sistema de coordenadas, con uno de sus vértices en el punto (0, 0, 0). Es un caso límite que asegura que el sistema puede manejar objetos con coordenadas mínimas en el origen.

El archivo de texto que corresponde al objeto que se cargó es el siguiente:

```

envolventeCubo3
1  env_Cubo_TercerOctante
2  8
3  -20 -20 -20
4  -20 -20 -10
5  -20 -10 -10
6  -20 -10 -20
7  -10 -20 -20
8  -10 -20 -10
9  -10 -10 -10
10 -10 -10 -20
11 4 0 1 2 3
12 4 4 5 6 7
13 4 0 1 5 4
14 4 1 2 6 5
15 4 2 3 7 6
16 4 3 0 4 7
17 -1
18

```

Figura 31. Contenido de la envolvente creada

En esta imagen, se puede observar que los valores esperados si corresponden a los generados por el programa, donde la coordenada mínima es -20,-20,-20 y la máxima es -10,-10,-10

Cubo en el origen

En este caso, el cubo está ubicado en el origen del sistema de coordenadas, con uno de sus vértices en el punto (0, 0, 0). Es un caso límite que asegura que el sistema puede manejar objetos con coordenadas mínimas en el origen.

El archivo de texto que corresponde al objeto que se cargó es el siguiente:

```

cuboorigen.txt
1  Cubo_EnElOrigen
2  8
3  0 0 0
4  1 0 0
5  1 1 0
6  0 1 0
7  0 0 1
8  1 0 1
9  1 1 1
10 0 1 1
11 4 0 1 2 3
12 4 4 5 6 7
13 4 0 1 5 4
14 4 1 2 6 5
15 4 2 3 7 6
16 4 3 0 4 7
17 -1

```

Figura 32. Contenido del objeto a cargar (cuboorigen.txt)

Valores esperados:

Mínimos (xmin, ymin, zmin): 0, 0, 0

Máximos (xmax, ymax, zmax): 1, 1, 1

Pasos por seguir:

1. Se comienza cargando el archivo Cubo_EnElOrigen.txt que contiene los datos del cubo.
2. Luego, se crea la caja envolvente usando el comando envolvente Cubo_EnElOrigen.
3. Para finalizar, se guarda la caja envolvente generada en el archivo resultado.txt con el comando guardar env_Cubo_EnElOrigen resultado.txt.

```

$ cargar cuboorigen.txt
(Resultado exitoso) El objeto Cubo_EnElOrigen ha sido ca
rgado exitosamente desde el archivo cuboorigen.txt.
El comando 'cargar' se ejecutó en 7.868e-05 segundos.
$ envolvente Cubo_EnElOrigen
(Resultado exitoso) La caja envolvente del objeto Cubo_E
nElOrigen se ha generado con el nombre env_Cubo_EnElOrig
en y se ha agregado a los objetos en memoria.
El comando 'envolvente' se ejecutó en 2.361e-05 segundos
.
$ guardar env_Cubo_EnElOrigen envolventeOrigen
(Resultado exitoso) La información del objeto env_Cubo_E
nElOrigen ha sido guardada exitosamente en el archivo en
volventeOrigen.
El comando 'guardar' se ejecutó en 0.00379901 segundos.

```

Figura 33. Ejecución de los comandos correspondientes


```

envolventeOrigen
1  env_Cubo_EnElOrigen
2  8
3  0 0 0
4  0 0 1
5  0 1 1
6  0 1 0
7  1 0 0
8  1 0 1
9  1 1 1
10 1 1 0
11 4 0 1 2 3
12 4 4 5 6 7
13 4 0 1 5 4
14 4 1 2 6 5
15 4 2 3 7 6
16 4 3 0 4 7
17 -1

```

Figura 34. Contenido al envoltente creado del objeto

En esta imagen, se puede observar que los valores esperados si corresponden a los generados por el programa, donde la coordenada mínima es 0,0,0 y la máxima es 1,1,1

Cubo con valores extremos

Este caso prueba un cubo con coordenadas que alcanzan los valores mínimos y máximos posibles en un sistema de doble precisión. Este escenario permite verificar cómo el sistema maneja valores límite y si se produce algún desbordamiento.

El archivo de texto que corresponde al objeto que se cargó es el siguiente:

```

cuboextremo.txt
1  Cubo_ValoresExtremos
2  8
3  -1.79769e+308 -1.79769e+308 -1.79769e+308
4  1.79769e+308 -1.79769e+308 -1.79769e+308
5  1.79769e+308 1.79769e+308 -1.79769e+308
6  -1.79769e+308 1.79769e+308 -1.79769e+308
7  -1.79769e+308 -1.79769e+308 1.79769e+308
8  1.79769e+308 -1.79769e+308 1.79769e+308
9  1.79769e+308 1.79769e+308 1.79769e+308
10 -1.79769e+308 1.79769e+308 1.79769e+308
11 4 0 1 2 3
12 4 4 5 6 7
13 4 0 1 5 4
14 4 1 2 6 5
15 4 2 3 7 6
16 4 3 0 4 7
17 -1
18

```

Figura 35. Contenido del objeto a cargar (cuboextremo.txt)

Valores esperados:

- Mínimos (xmin, ymin, zmin): -1.79769e+308, -1.79769e+308, -1.79769e+308
- Máximos (xmax, ymax, zmax): 1.79769e+308, 1.79769e+308, 1.79769e+308

```

$ cargar cuboextremo.txt
(Resultado exitoso) El objeto Cubo_ValoresExtremos ha sido
cargado exitosamente desde el archivo cuboextremo.txt
.
El comando 'cargar' se ejecutó en 0.00010782 segundos.
$ envolver Cubo_ValoresExtremos
(Resultado exitoso) La caja envolvente del objeto Cubo_ValoresExtremos
se ha generado con el nombre env_Cubo_ValoresExtremos y se ha agregado a los
objetos en memoria. El comando 'envolver' se ejecutó en 3.2669e-05 segundos.
$
$ guardar env_Cubo_ValoresExtremos envolventeExtremo
(Resultado exitoso) La información del objeto env_Cubo_ValoresExtremos ha sido
guardada exitosamente en el archivo envolventeExtremo.
El comando 'guardar' se ejecutó en 0.00114535 segundos.
$

```

Figura 36. Ejecución de los comandos correspondientes

```

1  env_Cubo_ValoresExtremos
2  8
3  -1.79769e+308 -1.79769e+308 -1.79769e+308
4  -1.79769e+308 -1.79769e+308 1.79769e+308
5  -1.79769e+308 1.79769e+308 1.79769e+308
6  -1.79769e+308 1.79769e+308 -1.79769e+308
7  1.79769e+308 -1.79769e+308 -1.79769e+308
8  1.79769e+308 -1.79769e+308 1.79769e+308
9  1.79769e+308 1.79769e+308 1.79769e+308
10 1.79769e+308 1.79769e+308 -1.79769e+308
11 4 0 1 2 3
12 4 4 5 6 7
13 4 0 1 5 4
14 4 1 2 6 5
15 4 2 3 7 6
16 4 3 0 4 7
17 -1

```

Figura 38. Contenido de la envolvente creada

En esta imagen, se puede observar que los valores esperados si corresponden a los generados por el programa, donde la coordenada mínima es $-1.79769e+308$, $-1.79769e+308$, $-1.79769e+308$ y la máxima es $1.79769e+308$, $1.79769e+308$, $1.79769e+308$.

2.4 Corrección Componente 1

En la primera revisión del proyecto, se encontraron problemas mínimos en su funcionamiento. El error más crítico se presentó durante el proceso de compilación en un entorno Ubuntu-Linux, ya que el código fuente se descargó originalmente desde la plataforma online Replit, lo que causó problemas debido a su forma particular de gestionar carpetas ocultas como Makefile o .gitignore, lo que provocó mensajes de error inesperados al trabajar en dicho entorno. Para solucionarlo, en la siguiente entrega se trabajó exclusivamente en una máquina virtual con Ubuntu-Linux, sin carpetas adicionales y

verificó que no hubiera problemas al compilar el código fuente en cada una de las máquinas del equipo de desarrollo.

Otro desafío surgió al intentar cargar un objeto con archivos de texto que contenían espacios o saltos de línea adicionales, lo que provocaba que el vector donde se almacenaban los vértices o caras creciera indefinidamente hasta ocupar la máxima cantidad de memoria disponible y generando un mensaje de error: '(Formato inválido) Las coordenadas de los vértices no son válidas en el archivo nombre_archivo.'. Afortunadamente, este proceso, al igual que otros, contaba con un manejo de errores que impedía que el programa se interrumpiera.

```
for (int i = 0; i < n_vertices; ++i) {
    if (std::getline(file, line)) {
        std::stringstream ss(line);
        double x, y, z;
        if (!(ss >> x >> y >> z)) {
            std::cout << "(Formato inválido) Las coordenadas de los vértices no son válidas
                          en el archivo " << nombre_archivo << "." << std::endl;
            return;
        }
        vertices.push_back(Vertex(x, y, z));
    } else {
        std::cout << "(Archivo vacío o incompleto) El archivo " << nombre_archivo << " no
                      contiene un objeto 3D válido." << std::endl;
        return;
    }
}
```

Figura 39. Sección de código defectuosa

```
Bienvenido al sistema de gestión de objetos. Digite 'ayuda' en caso de requerirlo.
$ cargar cube.txt
(Formato inválido) Las coordenadas de los vértices no son válidas en el archivo cube.txt.
```

Figura 40. Error en pantalla

Si se elimina la estructura de manejo de error, se observa cómo el error genera una instancia de extensión excedida al superar el límite máximo del vector.

```
Bienvenido al sistema de gestión de objetos. Digite 'ayuda' en caso de requerirlo.
$ cargar cube.txt
terminate called after throwing an instance of 'std::length_error'
  what():  cannot create std::vector larger than max_size()
Aborted (core dumped)
```

Figura 41. Error sin estructura de manejo

Puesto que la lectura del archivo era la causa raíz del problema, se desarrolló una función encargada de normalizar las entradas a un formato válido para que pueda ser procesada adecuadamente al cargar un objeto.

```
// Función para eliminar espacios en blanco y caracteres especiales al inicio y al final
void Menu::trim(std::string &s) {
    // Eliminar espacios y caracteres especiales al principio
    s.erase(s.begin(), std::find_if(s.begin(), s.end(), [](unsigned char c) {
        return !std::isspace(c) && c != '\n' && c != '\t' && c != '\r';
    }));
    // Eliminar espacios y caracteres especiales al final
    s.erase(std::find_if(s.rbegin(), s.rend(), [](unsigned char c) {
        return !std::isspace(c) && c != '\n' && c != '\t' && c != '\r';
    }).base(), s.end());
}
```

Figura 42. Función para normalización de entradas

Esta función se invoca cada vez que se lee una línea del archivo y recibe como parámetro la cadena de caracteres respectiva, haciendo que no ocurran problemas si un archivo no se encuentra totalmente bien escrito. Las pruebas realizadas consistieron en tratar de cargar nuevos objetos con archivos en un formato no estandarizado. A continuación, se presenta el resultado de una de ellas.

```
Cube
8
-1 -1 -1
1 -1 -1
    -1 1 -1
1 1 -1
-1 -1 1
1 -1 1
-1 1 1
1 1 1
    3 0 1 2
3 1 3 2
3 1 5 3
3 5 7 3
3 5 4 7
3 4 6 7
3 4 0 6
3 0 2 6
3 2 3 6
3 3 7 6
3 4 5 0
3 5 1 0
-1
```

Figura 43. Archivo con entradas no estandarizadas

```

for (int i = 0; i < n_vertices; ++i) {
    if (std::getline(file, line)) {
        trim(line);
        std::stringstream ss(line);
        double x, y, z;
        if (!(ss >> x >> y >> z)) {
            std::cout << "(Formato inválido) Las coordenadas de los
                        vértices no son válidas en el archivo "
                        << nombre_archivo << "." << std::endl;
            return;
        }
        vertices.push_back(Vertice(x, y, z));
    } else {
        std::cout << "(Archivo vacío o incompleto) El archivo "
                    << nombre_archivo << " no contiene un objeto 3D
                    válido." << std::endl;
        return;
    }
}

```

Figura 44. Implementación de sección de código con invocación a la función de estandarización

```

Bienvenido al sistema de gestión de objetos. Digite
'ayuda' en caso de requerirlo.
$ cargar cube.txt
(Resultado exitoso) El objeto Cube ha sido cargado
exitosamente desde el archivo cube.txt.

```

Figura 45. Resultado exitoso

Otro aspecto a considerar fue la impresión del listado de objetos cargados, ya que este no seguía el formato especificado en la función toString() del archivo objeto.cpp.

```

// Método para obtener una representación en cadena
std::string Objeto::toString() const {
    std::ostringstream oss;
    oss << "Nombre: " << nombre << "\n";
    oss << "Número de caras: " << caras.size() << "\n";
    for (const Cara& cara : caras) {
        oss << "Cara con " << cara.getNumeroVertices() << " vértices\n";
    }
    return oss.str();
}

```

Figura 46. Función en formato incorrecto

```
$ listado
Hay 1 objetos en memoria:
Nombre: Cube
Número de caras: 12
Cara con 3 vértices
Cara con 3 vértices
Cara con 3 vértices
Cara con 3 vértices
Cara con 3 vértices
Cara con 3 vértices
Cara con 3 vértices
Cara con 3 vértices
Cara con 3 vértices
Cara con 3 vértices
Cara con 3 vértices
Cara con 3 vértices
```

Figura 46. Salida con formato incorrecto

Puesto que esto solamente afecta la visualización de la información y no su contenido, la solución fue muy fácil, en la que lo único que cabe resaltar es que, al no contar con un TAD de aristas, se debió realizar cálculos adicionales para determinar su cantidad. Las pruebas sencillamente consistieron en cargar y listar múltiples objetos una vez corregida la función.

```

// Método para obtener una representación en cadena
std::string Objeto::toString() const {
    std::ostringstream flujo;
    std::vector<Vertice> vertices_unicos;
    int num_aristas = 0;

    for (int i = 0; i < caras.size(); ++i) {
        const std::vector<Vertice>& vertices_cara = caras[i].getVertices();
        for (int j = 0; j < vertices_cara.size(); ++j) {
            bool vertice_unico = true;
            for (int k = 0; k < vertices_unicos.size(); ++k) {
                if (vertices_unicos[k] == vertices_cara[j]) {
                    vertice_unico = false;
                    break;
                }
            }

            if (vertice_unico) {
                vertices_unicos.push_back(vertices_cara[j]);
            }

            Vertice vertice_actual = vertices_cara[j];
            Vertice vertice_siguiente = vertices_cara[(j + 1) % vertices_cara.size()];
            bool arista_duplicada = false;

            for (int m = 0; m < i; ++m) {
                const std::vector<Vertice>& vertices_cara_anterior = caras[m].getVertices();
                for (int n = 0; n < vertices_cara_anterior.size(); ++n) {
                    Vertice v1_anterior = vertices_cara_anterior[n];
                    Vertice v2_anterior = vertices_cara_anterior[(n + 1) % vertices_cara_anterior.size()];

                    if ((vertice_actual == v1_anterior && vertice_siguiente == v2_anterior) ||
                        (vertice_actual == v2_anterior && vertice_siguiente == v1_anterior)) {
                        arista_duplicada = true;
                        break;
                    }
                }

                if (arista_duplicada) {
                    break;
                }
            }

            if (!arista_duplicada) {
                num_aristas++;
            }
        }
    }

    flujo << nombre << " contiene " << vertices_unicos.size() << " vertices, "
        << num_aristas << " aristas y " << caras.size() << " caras.";

    return flujo.str();
}

```

Figura 47. Función toString() corregida

```

Bienvenido al sistema de gestión de objetos. Digite 'ayuda' en caso de requerirlo.
$ cargar cube.txt
(Resultado exitoso) El objeto Cube ha sido cargado exitosamente desde el archivo cube.txt.
$ cargar pyramid.txt
(Resultado exitoso) El objeto Pyramid ha sido cargado exitosamente desde el archivo pyramid.txt.
$ cargar octahedron.txt
(Resultado exitoso) El objeto Octahedron ha sido cargado exitosamente desde el archivo octahedron.txt.
$ cargar rotated_rectangular_prism.txt
(Resultado exitoso) El objeto Rotated_Rectangular_Prism ha sido cargado exitosamente desde el archivo
rotated_rectangular_prism.txt.
$ listado
Hay 4 objetos en memoria:
Cube contiene 8 vertices, 18 aristas y 12 caras.
Pyramid contiene 6 vertices, 13 aristas y 9 caras.
Octahedron contiene 6 vertices, 12 aristas y 8 caras.
Rotated_Rectangular_Prism contiene 8 vertices, 12 aristas y 6 caras.

```

Figura 48. Impresión con el formato correcto

Es importante señalar que el archivo cube.txt no representa un cubo real, y, por lo tanto, sus valores de vértices, aristas y caras son inconsistentes. Esto se debe a que el archivo contiene varias caras triangulares en lugar de cuadradas, lo que genera una representación de un cubo con el doble de caras.

Un desacierto similar estuvo en el nombramiento de la caja de envoltorio global, pues esta se llamó “caja_global” en lugar de “env_global”.

```
Objeto cajaGlobal("caja_global");
```

Figura 49. Nombre incorrecto

Lógicamente, se cambió el nombre de la variable y se hizo la comprobación de que esta se imprimiera correctamente. No se realizaron más pruebas debido a la simplicidad de la corrección.

```
Objeto cajaGlobal("env_global");
```

Figura 50. Nombre correcto

```
Bienvenido al sistema de gestión de objetos. Digite 'ayuda' en caso de requerirlo.
$ cargar octahedron(1).txt
(Resultado exitoso) El objeto Octahedron ha sido cargado exitosamente desde el archivo
octahedron(1).txt.
El comando 'cargar' se ejecutó en 8.22e-05 segundos.
$ envolver
(Resultado exitoso) La caja de ajuste global se ha generado con el nombre env_global y
se ha agregado a los objetos en memoria.
El comando 'envolver' se ejecutó en 4.435e-05 segundos.
```

Figura 51. Salida correcta

Como se aprecia en la imagen, se incluyó también una métrica del tiempo de ejecución de cada comando. Esto se hizo iniciando un cronómetro luego de que la función manejarEntradaUsuario() reciba el comando del usuario y parándolo luego de la ejecución de dicho comando. En cuanto a correcciones del presente documento, no hubo ningún error o mejora a implementar.

3 Componente 2: vértices más cercanos

El segundo componente del proyecto se enfoca en la implementación de algoritmos que permiten identificar los vértices más cercanos en los objetos tridimensionales cargados en memoria. Este módulo amplía las capacidades desarrolladas en el Componente 1, al introducir funciones que facilitan la búsqueda y localización eficiente de puntos clave en las mallas poligonales.

3.1 Descripción de los comandos

v_cercano px py pz nombre_objeto

El comando *v_cercano* está diseñado para identificar el vértice del objeto tridimensional, identificado por *nombre_objeto*, más cercano a un punto dado en el espacio, especificado por las coordenadas *px*, *py* y *pz*. Este proceso se lleva a cabo utilizando la distancia euclidiana para comparar la proximidad de cada vértice del objeto al punto de referencia proporcionado por el usuario.

La ejecución del comando comienza verificando si el objeto indicado (*nombre_objeto*) se ha cargado en memoria previamente. El comando finalizará de inmediato si el objeto no está disponible y se mostrará un mensaje de error que indica que el objeto no ha sido encontrado. Esto asegura que solo se trabajará con objetos válidos y previamente cargados.

Posteriormente, se construye un KD-Tree (un tipo de árbol espacial que optimiza la búsqueda de puntos cercanos) con los vértices del objeto una vez que se confirma la existencia del objeto en la lista de objetos cargados. Este KD-Tree reduce el tiempo necesario para encontrar el vértice más cercano, incluso en objetos con muchos vértices.

Para la construcción del KD-Tree tomemos como ejemplo este cubo con sus respectivos vértices:

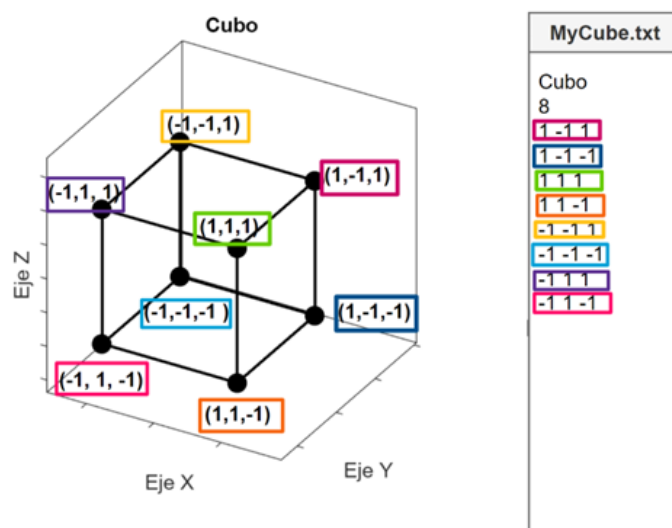


Figura 52. Cubo para la construcción de KD-Tree

El primer paso es elegir la raíz del árbol, que puede ser cualquiera de los vértices del cubo. En este ejemplo, usaremos el vértice (1,1,1) como la raíz.

Luego, se procede a dividir los puntos por niveles, usando un eje distinto en cada nivel para dividir los puntos. El primer nivel corresponde al eje X, el segundo nivel al eje Y, y el tercer nivel al eje Z.

1. **Primer nivel (eje X):** Se dividen los puntos en dos grupos según su coordenada X.
 - En el primer grupo se colocan los puntos cuya X sea menor o igual a 1. Para este caso, los puntos son: (-1, 1, 1), (-1, 1, -1), (-1, -1, 1), (-1, -1, -1).
 - En el segundo grupo se colocan los puntos cuya X sea mayor a 1. Para este caso, los puntos son: (1, 1, -1), (1, -1, 1), (1, -1, -1).
2. **Segundo nivel (eje Y):** Para cada grupo, se dividen nuevamente los puntos, esta vez usando el eje Y.
 - Para el grupo izquierdo ($X \leq 1$), se coloca como raíz del subárbol izquierdo el vértice (-1, 1, 1). Luego, los puntos se dividen según su coordenada Y:
 - A la izquierda, los puntos cuya Y es menor o igual a 1: (-1, -1, 1), (-1, -1, -1).
 - A la derecha, los puntos cuya Y es mayor a 1: (-1, 1, -1).
 - Para el grupo derecho ($X > 1$), se coloca como raíz del subárbol derecho el vértice (1, 1, -1). Luego, los puntos se dividen según su coordenada Y:
 - A la izquierda, los puntos cuya Y es menor o igual a 1: (1, -1, 1), (1, -1, -1).
3. **Tercer nivel (eje Z):** Finalmente, para cada subgrupo, se realiza la última división según el eje Z.
 - Para los puntos (-1, -1, 1) y (-1, -1, -1), se coloca (-1, -1, 1) como raíz del subárbol y (-1, -1, -1) como su hijo izquierdo.
 - Para los puntos (1, -1, 1) y (1, -1, -1), se coloca (1, -1, 1) como raíz del subárbol y (1, -1, -1) como su hijo izquierdo.

El resultado final del árbol es el siguiente:

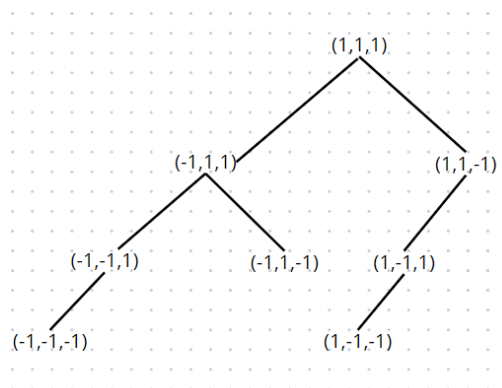


Figura 53. KD-Tree del cubo

Luego, utilizando el KD-Tree, el comando busca el vértice más cercano al punto (px, py, pz) . Si se encuentra un vértice adecuado, el sistema calculará la distancia mínima entre el punto dado y el vértice encontrado. Además, se identificará el índice del vértice en el objeto y se mostrarán sus coordenadas exactas junto con la distancia calculada.

En caso de que no se encuentre un vértice cercano o si el KD-Tree no puede procesar el objeto adecuadamente, el comando finalizará con un mensaje de error. Este escenario es poco frecuente, pero se maneja para garantizar la robustez del sistema.

v_cercano px py pz

Este comando busca el vértice más cercano, en términos de distancia euclidiana, a un punto dado por las coordenadas px , py y pz , entre todos los objetos cargados en memoria. El proceso que se lleva a cabo es similar al anterior, en primer lugar, el comando verifica si hay objetos disponibles; si no hay objetos cargados, muestra un mensaje indicando que no se puede realizar la búsqueda.

En el caso de que, si existan objetos en memoria, el comando procede a crear un KD-Tree y se inserta cada vértice de todos los objetos en este árbol. Esto se logra iterando sobre cada objeto, sus caras y los vértices de estas caras, y almacenando cada vértice junto con el nombre del objeto al que pertenece.

Luego, se define el punto objetivo usando las coordenadas proporcionadas por el usuario una vez que todos los vértices están en el KD-Tree. Después de eso, el comando utiliza el KD-Tree para encontrar el nodo que contiene el vértice más cercano a este punto. Si se encuentra un vértice cercano, el comando calcula la distancia euclidiana entre el vértice encontrado y el punto objetivo.

Además, se busca el índice del vértice más cercano dentro de los objetos cargados. Esto se logra rastreando cada objeto y sus respectivas caras para determinar en qué cara y posición se encuentra el vértice más cercano.

Finalmente, el comando imprime en pantalla el resultado de la búsqueda. Si el vértice pertenece a un solo objeto, se mostrará el índice del vértice, sus coordenadas y la distancia al punto objetivo. En el caso de que el vértice se encuentre en más de un objeto, se enumerarán todos los objetos correspondientes. Si no se encuentra ningún vértice cercano, se informará al usuario sobre este error.

v_cercanos_caja nombre_objeto

El comando *v_cercanos_caja nombre_objeto* se utiliza para identificar los vértices del objeto especificado, denominado *nombre_objeto*, que son más cercanos a los puntos (vértices) que definen la caja envolvente del objeto. Primero, el comando verifica si el objeto está cargado en la memoria. Si el objeto no existe, se notifica al usuario que el objeto no ha sido cargado y se termina la ejecución del comando.

A continuación, se verifica si la caja envolvente ya se ha creado si el objeto está disponible. En caso de que no esté disponible, se procede a calcularla. Luego, se busca el objeto correspondiente en la lista de objetos cargados una vez que se ha confirmado la existencia de la caja envolvente. Si la caja no se encuentra, se informa al usuario de un error en su creación o búsqueda.

La siguiente etapa implica la creación de dos vectores, uno para almacenar los vértices distintivos del objeto original y otro para sus respectivos índices. Se recorren las caras del objeto y se extraen sus vértices a través de un bucle, asegurándose de que cada vértice se inserte una sola vez en el vector de vértices únicos mientras se almacena su índice en el vector de índices.

Luego, el comando crea la variable *esquinas*, que incluirá los vértices que representan las esquinas de la caja envolvente. Estos vértices se extraen de la lista de caras de la caja envolvente y se almacenarán sin duplicados.

Una vez que se han recopilado todos los vértices de las esquinas de la caja envolvente, el comando imprime en pantalla una tabla que muestra el resultado de la búsqueda. La tabla incluye las coordenadas de cada esquina de la caja envolvente, el índice del vértice más cercano, sus coordenadas actuales y la distancia entre ellos. Para cada esquina, se calcula la distancia euclidiana al verificar cada vértice único del objeto, y se identifica cuál de ellos es el más cercano.

Finalmente, si se creó una caja envolvente durante la ejecución del comando, esta se descarga de la memoria para evitar mantener objetos innecesarios. En caso contrario, la información sobre los vértices cercanos se presenta de manera clara y estructurada, proporcionando al usuario una visión detallada de las relaciones espaciales entre los vértices del objeto y las esquinas de su caja envolvente.

3.2 Diseño

En esta subsección, se presentará el diseño detallado del sistema, el cual es de carácter acumulativo, reflejando fielmente su evolución a lo largo del proceso de desarrollo. A lo largo de este apartado, se describirán los Tipos Abstractos de Datos (TADs) empleados y las interacciones entre ellos, asegurando una representación precisa y completa del sistema en su estado actual.

Descripción de los Tipos Abstractos de Datos (TADs):

En la primera entrega del proyecto se propusieron los TADs menu, objeto, cara y vértice. Dado que estos no han experimentado modificaciones en su diseño o funcionalidad, no serán descritos nuevamente en este documento, a excepción del TAD menu, al cual se le ha añadido la función trim, que facilita la manipulación de cadenas de texto eliminando espacios innecesarios al inicio y al final de las mismas.

En esta segunda entrega, se han introducido dos nuevos TADs: NodoKdTree y KdTree, diseñados con el propósito de mejorar el rendimiento en la identificación de vértices cercanos dentro de objetos tridimensionales. El TAD NodoKdTree representa un nodo dentro de un KdTree, que es una estructura de datos especializada en la búsqueda eficiente de puntos en espacios de múltiples dimensiones. Por su parte, el TAD KdTree gestiona el árbol en su totalidad, permitiendo realizar consultas rápidas sobre la proximidad entre vértices y un punto dado o los vértices más cercanos a la s esquinas de la caja envolvente de un objeto.

NodoKdTree

El TAD NodoKdTree es parte de la estructura de datos KdTree, que organiza puntos en el espacio tridimensional para búsquedas eficientes. Cada nodo contiene un vértice y referencias a hijos izquierdo y derecho.

Estado:

- punto_asociado: (punto), objeto, vértice representado por el nodo.
- nombres_de_objetos: (nombresObjetos), plantilla, lista de nombres de objetos asociados al vértice.
- hijo_izquierdo: (izquierda), puntero, apunta al nodo hijo izquierdo en el árbol.
- hijo_derecho: (derecha), puntero, apunta al nodo hijo derecho en el árbol.
- nivel_de_profundidad: (profundidad), numérico, indica el nivel de profundidad del nodo dentro del árbol.

Interfaz:

- `NodoKDTree ()`, constructor por defecto, crea un nodo vacío con profundidad cero y sin hijos.
- `NodoKDTree (punto, nombreObjeto, profundidad)`, crea un nodo con un vértice (punto), un objeto (nombreObjeto) y una profundidad (profundidad).
- `getPunto()`: punto, retorna el vértice asociado al nodo.
- `setPunto(punto)`, asigna un nuevo vértice al nodo.
- `getIzquierda()`: izquierda, retorna el puntero al nodo hijo izquierdo.
- `setIzquierda(nodo)`, establece el puntero al nodo hijo izquierdo.
- `getDerecha()`: derecha, retorna el puntero al nodo hijo derecho.
- `setDerecha(nodo)`, establece el puntero al nodo hijo derecho.
- `getProfundidad()`: profundidad, retorna la profundidad del nodo en el árbol.
- `setProfundidad(profundidad)`, establece la profundidad del nodo.
- `getNombresObjetos()`: nombresObjetos, retorna los nombres de los objetos asociados al vértice del nodo.
- `agregarNombreObjeto(nombreObjeto)`, añade un nuevo nombre de objeto a la lista del nodo.
 - Pre: El nombre del objeto no debe estar ya en la lista.
- `operator==(otro)`: booleano, compara dos nodos para verificar si contienen el mismo vértice.
 - Pre: Ambos nodos deben estar definidos.
- `~NodoKDTree()`, destruye el nodo y libera los recursos asociados a sus hijos.

KDTree

El TAD `KDTree` organiza los puntos en un espacio tridimensional utilizando un árbol de partición espacial. Permite la inserción de vértices y la búsqueda eficiente del nodo más cercano a un punto dado.

Estado

- `raiz`: (raiz), puntero, apunta a la raíz del árbol `KDTree`.

Interfaz

- `KDTree()`, crea un árbol KD vacío sin nodos.
- `insertar (punto, nombreObjeto)`, inserta un nuevo vértice en el árbol con su respectivo nombre de objeto.
- `encontrarMasCercano(punto)`: `NodoKDTree*`, retorna el nodo más cercano al vértice dado.
- `~KDTree()`, destruye el árbol y libera la memoria asociada.

Menu

El TAD menu gestiona las operaciones relacionadas con la carga, manipulación y almacenamiento de objetos tridimensionales, así como la interacción con los usuarios a través de un menú. El menu organiza los objetos cargados en memoria y permite ejecutar una serie de operaciones para facilitar la construcción y análisis de modelos 3D.

Estado

- objetos_cargados: (objetosCargados), lista, almacena los objetos tridimensionales que han sido cargados en memoria.

Interfaz

- Menu(), inicializa un menú vacío con la capacidad de gestionar objetos.
- objetoCargado(nombre_archivo): bool, verifica si un objeto con un nombre dado ya ha sido cargado en memoria.
 - Post: Retorna verdadero si el objeto ya está cargado, falso en caso contrario.
- mostrarMenu(), muestra las opciones del menú al usuario.
- manejarEntradaUsuario(), gestiona y procesa la entrada del usuario desde el menú.
- mostrarAyuda(), muestra una ayuda sobre cómo utilizar el sistema.
- salirPrograma(), cierra el programa.
- trim(s), elimina espacios y caracteres especiales al principio y al final de una cadena de caracteres.
- cargarObjeto(nombre_archivo), carga un objeto desde un archivo y lo almacena en memoria.
 - Pre: nombre_archivo debe ser una cadena válida que apunte a un archivo existente que no haya sido cargado previamente.
- listarObjetosCargados(), muestra una lista de todos los objetos que están actualmente cargados en memoria.
- calcularCajaEnvolvente(nombre_objeto), calcula y muestra la caja envolvente de un objeto específico.
 - Pre: nombre_objeto debe referirse a un objeto cargado en memoria.
- crearCajaDeAjusteGlobal(), calcula y muestra la caja envolvente que incluye todos los objetos cargados en memoria.
 - Pre: deben existir objetos cargados en memoria.
- descargarObjeto(nombre_objeto), elimina un objeto específico de la memoria.
 - Pre: nombre_objeto debe referirse a un objeto cargado en memoria.
- guardarObjetoEnArchivo(nombre_objeto, nombre_archivo), guarda un objeto cargado en memoria en un archivo.
 - Pre: nombre_objeto debe referirse a un objeto cargado en memoria; nombre_archivo debe ser una cadena válida.
- verticeCercanoObjeto(px, py, pz, nombre_objeto), encuentra y muestra el vértice más cercano a un punto específico dentro de un objeto.

- Pre: px, py, pz deben ser valores numéricos que representan el punto de interés; nombre_objeto debe referirse a un objeto cargado en memoria.
- verticeCercanoTodosObjetos(px, py, pz), encuentra y muestra el vértice más cercano a un punto específico considerando todos los objetos cargados en memoria.
 - Pre: px, py, pz deben ser valores numéricos que representan el punto de interés.
- verticesCercanosCaja(nombre_objeto), muestra todos los vértices cercanos a la caja envolvente de un objeto específico.
 - Pre: nombre_objeto debe referirse a un objeto cargado en memoria.
- rutaCortaVertices(v1, v2, nombre_objeto), calcula y muestra la ruta más corta entre dos vértices dentro de un objeto específico.
 - Pre: v1 y v2 deben ser índices válidos de vértices dentro de nombre_objeto.
- rutaCortaCentro(v1, nombre_objeto), calcula y muestra la ruta más corta entre un vértice y el centro del objeto.
 - Pre: v1 debe ser un índice válido de un vértice dentro de nombre_objeto.

Diagrama de Relación de TADs:

El diagrama de relación de TADs de la segunda entrega muestra la estructura y conexión entre los diferentes Tipos Abstractos de Datos (TADs) diseñados para la gestión y manipulación de objetos tridimensionales. Los TADs principales son Vertice, Cara, Objeto, Menu, NodoKdTree y KdTree, y cada uno tiene un rol específico en el sistema.

El TAD Vertice representa un punto en el espacio tridimensional mediante sus coordenadas (x, y, z), y es la base para el TAD Cara, que utiliza un conjunto de vértices para formar una superficie plana. Estas caras se agrupan dentro del TAD Objeto, que modela un objeto tridimensional completo. Por su parte, el TAD Menu proporciona una interfaz para que el usuario interactúe con el sistema, permitiendo operaciones como cargar, listar y gestionar objetos en memoria.

En cuanto a los TADs NodoKdTree y KdTree, estos implementan una estructura de árbol espacial que optimiza la búsqueda de vértices cercanos. Es importante señalar que, aunque el NodoKdTree almacena nombres de los objetos a los que pertenecen los vértices, no hay una relación directa entre NodoKdTree y Objeto, ya que solo se copian los nombres de los objetos. En cambio, la relación directa está en los vértices, ya que son los que efectivamente se usan en el árbol para optimizar las búsquedas.

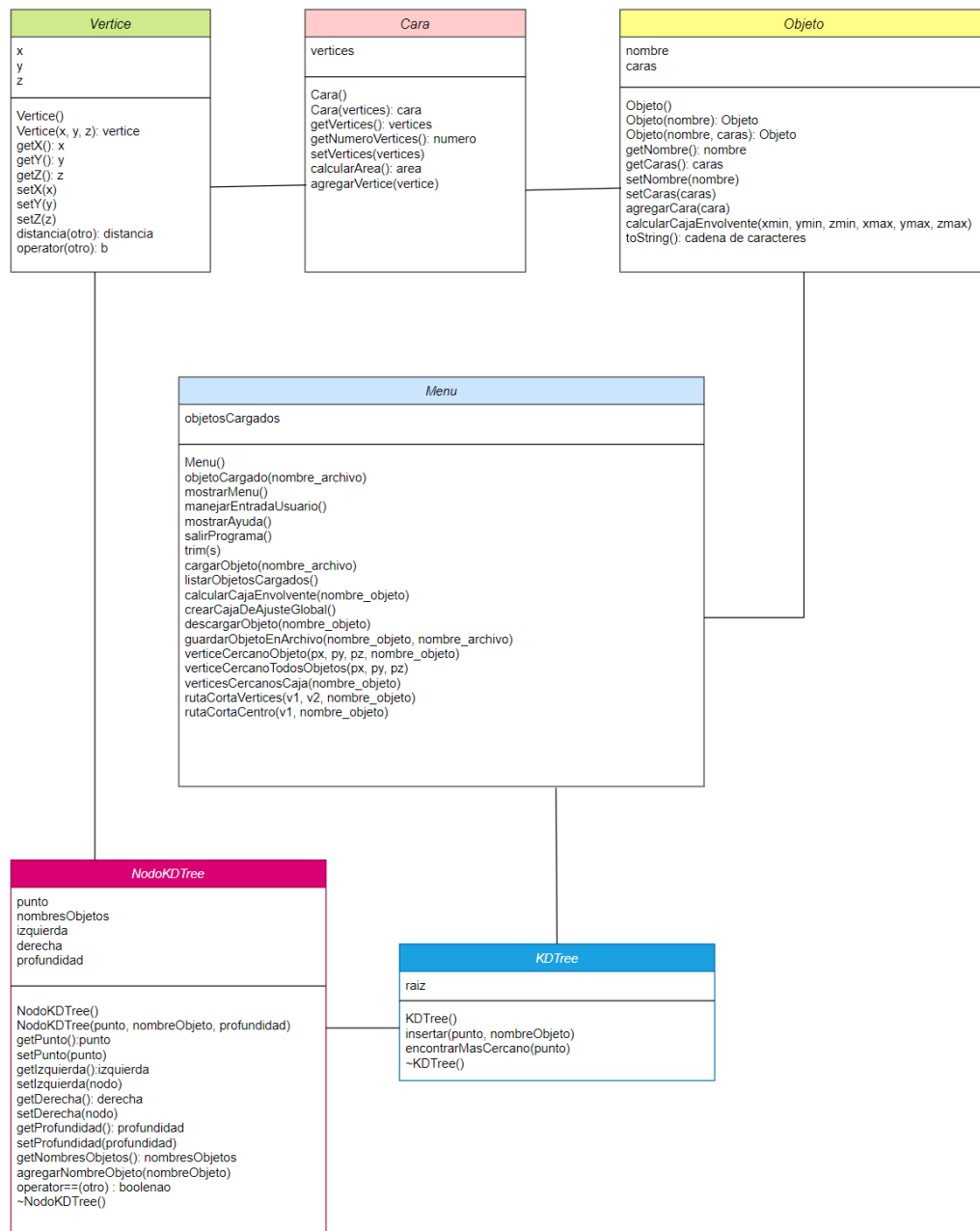


Figura 54. Diagrama de relación de TADs

3.3 Plan de pruebas

Para el diseño del plan de pruebas, se consideró únicamente el comando de `v_cercanos_caja`. Para ello, se planteó calcular manualmente los vértices cercanos a cada caja, la distancia respectiva y compararlo con el resultado de la función. La primera prueba se realizó en un prisma rectangular, correspondiente al siguiente archivo de texto que contiene este objeto.

```

1 Rectangular_Prism
2 8
3 0 0 0
4 10 0 0
5 10 1 0
6 0 1 0
7 0 0 1
8 10 0 1
9 10 1 1
10 0 1 1
11 4 0 1 2 3
12 4 4 5 6 7
13 4 3 2 6 7
14 4 0 1 5 4
15 4 0 3 7 4
16 4 1 2 6 5
17 -1

```

Figura 55. Archivo de prisma rectangular

En este caso, se esperaba que, por tratarse de un prisma rectangular, los vértices de la caja coincidan con los del objeto, y, por lo tanto, su distancia euclidiana sea 0. Al ejecutar el comando, `v_cercanos_caja`, la salida se observa de la siguiente manera.

```

(Resultado exitoso) Los vertices del objeto Rectangular_Prism más cercanos a las esquinas de su caja envolvente son:
Esquina      Vertice      Distancia
-----
1 (0,0,0) -> 0 (0,0,0) -> 0
2 (0,0,1) -> 4 (0,0,1) -> 0
3 (0,1,1) -> 7 (0,1,1) -> 0
4 (0,1,0) -> 3 (0,1,0) -> 0
5 (10,0,0) -> 1 (10,0,0) -> 0
6 (10,0,1) -> 5 (10,0,1) -> 0
7 (10,1,1) -> 6 (10,1,1) -> 0
8 (10,1,0) -> 2 (10,1,0) -> 0
-----
El comando 'v_cercanos_caja' se ejecutó en 7.5471e-05 segundos.

```

Figura 56. Resultado de `v_cercanos_caja` para prisma rectangular

Para la segunda prueba, se analizó un dodecaedro con la siguiente estructura, junto con los valores calculados.

```

1  Dodecahedron
2  20
3  1 1 1
4  1 1 -1
5  1 -1 1
6  1 -1 -1
7  -1 1 1
8  -1 1 -1
9  -1 -1 1
10 -1 -1 -1
11 0 0.618 1.618
12 0 0.618 -1.618
13 0 -0.618 1.618
14 0 -0.618 -1.618
15 0.618 1.618 0
16 0.618 -1.618 0
17 -0.618 1.618 0
18 -0.618 -1.618 0
19 1.618 0 0.618
20 1.618 0 -0.618
21 -1.618 0 0.618
22 -1.618 0 -0.618
23 5 0 8 10 2 16
24 5 1 9 11 3 17
25 5 4 8 10 6 18
26 5 5 9 11 7 19
27 5 12 0 2 14 4
28 5 13 1 3 15 5
29 5 16 8 4 14 12
30 5 17 9 5 15 13
31 5 18 10 6 19 4
32 5 19 11 7 15 5
33 5 0 12 13 1 16
34 5 3 17 13 7 15
35 5 6 18 14 4 19
36 5 2 16 12 14 10

```

Figura 57. Archivo de dodecaedro

Esquina (Caja)	Vértice (Dodecaedro)	Distancia
$(-1.618, -1.618, -1.618)$	$(-1.0, -1.0, -1.0)$	1.070407
$(-1.618, -1.618, 1.618)$	$(-1.0, -1.0, 1.0)$	1.070407
$(-1.618, 1.618, -1.618)$	$(-1.0, 1.0, -1.0)$	1.070407
$(-1.618, 1.618, 1.618)$	$(-1.0, 1.0, 1.0)$	1.070407
$(1.618, -1.618, -1.618)$	$(1.0, -1.0, -1.0)$	1.070407
$(1.618, -1.618, 1.618)$	$(1.0, -1.0, 1.0)$	1.070407
$(1.618, 1.618, -1.618)$	$(1.0, 1.0, -1.0)$	1.070407
$(1.618, 1.618, 1.618)$	$(1.0, 1.0, 1.0)$	1.070407

Tabla 1. Valores esperados del dodecaedro

La salida al ejecutar el comando en este objeto es la siguiente.

```
(Resultado exitoso) Los vertices del objeto Dodecahedron más cercanos a las esquinas de su caja envolvente son:
Esquina      Vertice      Distancia
-----
1  (-1.618,-1.618,-1.618)  -> 14 (-1,-1,-1)  -> 1.07041
2  (-1.618,-1.618,1.618)   -> 11 (-1,-1,1)   -> 1.07041
3  (-1.618,1.618,1.618)    -> 10 (-1,1,1)    -> 1.07041
4  (-1.618,1.618,-1.618)   -> 13 (-1,1,-1)   -> 1.07041
5  (1.618,-1.618,-1.618)    -> 8 (1,-1,-1)    -> 1.07041
6  (1.618,-1.618,1.618)    -> 3 (1,-1,1)     -> 1.07041
7  (1.618,1.618,1.618)     -> 0 (1,1,1)      -> 1.07041
8  (1.618,1.618,-1.618)    -> 5 (1,1,-1)     -> 1.07041
El comando 'v_cercanos_caja' se ejecutó en 7.608e-05 segundos.
```

Figura 58. Resultado de dodecaedro

Se observa que los datos concuerdan hasta la quinta cifra decimal. En la tercera prueba se utilizó un objeto titulado como cube, que como se mencionó en la sección de corrección del componente 1, se trata de un cubo donde cada cara normal se representa por dos triangulares, el cual tiene la siguiente estructura.

```
1  Cube
2  8
3  -1 -1 -1
4  1 -1 -1
5  -1 1 -1
6  1 1 -1
7  -1 -1 1
8  1 -1 1
9  -1 1 1
10 1 1 1
11 3 0 1 2
12 3 1 3 2
13 3 1 5 3
14 3 5 7 3
15 3 5 4 7
16 3 4 6 7
17 3 4 0 6
18 3 0 2 6
19 3 2 3 6
20 3 3 7 6
21 3 4 5 0
22 3 5 1 0
23 -1
```

Figura 59. Archivo de cubo

Nuevamente, por tratarse de un objeto cuya caja envolvente es igual a él, se espera obtener vértices y esquinas iguales con distancia 0. El resultado exitoso al ejecutar el comando en este objeto se observa a continuación.

```

(Resultado exitoso) Los vertices del objeto Cube más cercanos a las esquinas de su caja envolvente son:
Esquina      Vertice      Distancia
-----
1  (-1,-1,-1) -> 0  (-1,-1,-1) -> 0
2  (-1,-1,1)  -> 6  (-1,-1,1)  -> 0
3  (-1,1,1)   -> 7  (-1,1,1)   -> 0
4  (-1,1,-1)  -> 2  (-1,1,-1)  -> 0
5  (1,-1,-1)  -> 1  (1,-1,-1)  -> 0
6  (1,-1,1)   -> 4  (1,-1,1)   -> 0
7  (1,1,1)    -> 5  (1,1,1)    -> 0
8  (1,1,-1)   -> 3  (1,1,-1)   -> 0
-----
El comando 'v_cercanos_caja' se ejecutó en 6.203e-05 segundos.

```

Figura 60. Resultado de cubo

En la cuarta y última prueba, se utilizó un prisma rectangular rotado con la siguiente estructura. Por la complejidad del objeto, se presenta a continuación, además, la representación 3D de la figura y su caja envolvente.

```

1  Rotated_Rectangular_Prism
2  8
3  -8.66 -1.1 -0.5
4  0.5 -1.1 -8.66
5  0.5 0.1 -8.66
6  -8.66 0.1 -0.5
7  -8.66 -1.1 0.5
8  0.5 -1.1 0.5
9  0.5 0.1 0.5
10 -8.66 0.1 0.5
11 4 0 1 2 3
12 4 4 5 6 7
13 4 3 2 6 7
14 4 0 1 5 4
15 4 0 3 7 4
16 4 1 2 6 5
17 -1

```

Figura 61. Archivo de prisma rectangular invertido

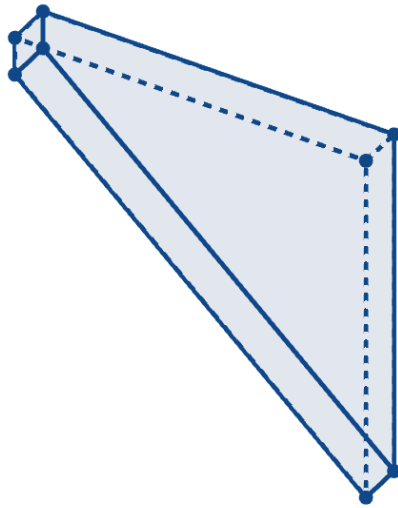


Figura 62. Representación 3D de prisma rectangular invertido

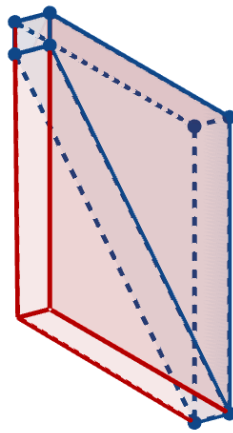


Figura 63. Representación 3D de caja envolvente

La anterior imagen muestra que, de los 8 vértices del objeto, solo 2 no coinciden con su caja envolvente, siendo que estos dos vértices serán los más cercanos a las esquinas restantes. Para una mejor visualización, se presenta una imagen con los vértices del objeto en azul y la caja envolvente.

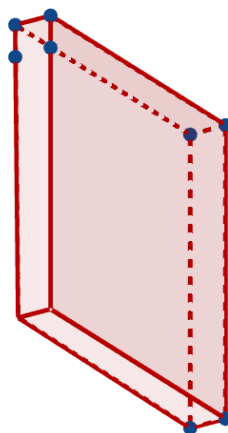


Figura 64. Visualización de vértices y esquinas

Finalmente, se determina que las dos distancias a calcular se obtienen restando la coordenada z del vértice del objeto a la coordenada z de la esquina de la caja, ya que las demás coordenadas son iguales, como se demostró anteriormente. Además, la distancia debe ser la misma, dado que entre los dos vértices solo cambia la coordenada y. Por lo tanto, se tiene:

$$\text{Distancia} = |-8.66 - (-0.5)| = 8.16.$$

Al ejecutar el comando en este objeto, el resultado observado es el siguiente, el cual coincide con el análisis realizado.

```
(Resultado exitoso) Los vertices del objeto Rotated_Rectangular_Prism más cercanos a las esquinas de su caja
a envolvente son:
Esquina      Vertice      Distancia
-----
1  (-8.66,-1.1,-8.66)  -> 0  (-8.66,-1.1,-0.5)  -> 8.16
2  (-8.66,-1.1,0.5)   -> 4  (-8.66,-1.1,0.5)   -> 0
3  (-8.66,0.1,0.5)    -> 7  (-8.66,0.1,0.5)    -> 0
4  (-8.66,0.1,-8.66)  -> 3  (-8.66,0.1,-0.5)   -> 8.16
5  (0.5,-1.1,-8.66)   -> 1  (0.5,-1.1,-8.66)   -> 0
6  (0.5,-1.1,0.5)     -> 5  (0.5,-1.1,0.5)     -> 0
7  (0.5,0.1,0.5)      -> 6  (0.5,0.1,0.5)      -> 0
8  (0.5,0.1,-8.66)    -> 2  (0.5,0.1,-8.66)    -> 0
El comando 'v_cercanos_caja' se ejecutó en 0.00011908 segundos.
```

Figura 65. Resultado de prisma rectangular rotado

En este orden de ideas, la función `v_cercanos_caja` funciona con una alta precisión y coincide con los valores empíricos calculados en las pruebas.

3.4 Corrección Componente 2

Con respecto al diseño anterior, no se presentaron errores en la ejecución y pruebas de este. No obstante, al realizar el diseño del componente 3, se notó la necesidad de guardar de alguna forma el orden de los vértices para que, al ingresarlos en los comandos, se mantuvieran iguales los índices al ingresarlos con respecto a su posición al realizar la lectura de cada archivo. Por ello, se tomó la decisión de crear una clase de Objeto un vector de vértices y crear su getter y setter.

```
class Objeto {
private:
    std::string nombre;
    std::vector<Cara> caras;
    std::vector<Vertice> verticesTotales;

public:
    // Constructor por defecto
    Objeto();

    // Constructor con nombre
    Objeto(const std::string& nombre);

    // Constructor con parámetros
    Objeto(const std::string& nombre, const std::vector<Vertice>& vertices, const std::vector<Cara>& caras);

    // Getters
    std::string getNombre() const;
    const std::vector<Cara>& getCaras() const;
    const std::vector<Vertice>& getVertices() const;

    // Setters
    void setNombre(const std::string& nombre);
    void setCaras(const std::vector<Cara>& caras);
    void setVertices(const std::vector<Vertice>& vertices);
};
```

Figura 66. Nueva clase Objeto.

Aunque esta solución resuelve el problema de manera efectiva, no es necesariamente la más eficiente desde el punto de vista de la memoria o el rendimiento. Por ejemplo, una alternativa más eficiente podría haber sido modificar la clase Vertice para almacenar directamente su índice como un atributo. De esta manera, no habría necesidad de mantener un vector adicional en la clase Objeto, lo que habría reducido el uso de memoria. Otras posibles soluciones podrían haber involucrado la implementación de estructuras de datos más avanzadas que no requirieran almacenar un vector adicional, pero estas alternativas habrían supuesto un mayor esfuerzo en términos de complejidad y tiempo de desarrollo.

Solo fue necesario modificar la función cargarObjeto() para fijar el nuevo vector con el valor de uno temporal que ya almacenaba los vértices en orden, justo al final de la función. Esta implementación no solo resuelve el problema de mantener el orden de los vértices, sino que también evita una modificación extensiva de otras partes del código, como los árboles o la estructura de datos que manejan los vértices.


```
objeto.setVertices(vertices);
objetosCargados.push_back(objeto);
```

Figura 67. Agregar vértices a verticesTotales.

4 Componente 3: ruta más corta

El tercer componente del proyecto se centra en el desarrollo de funciones avanzadas para la manipulación de mallas poligonales, enfocándose en la identificación de rutas más cortas entre vértices y hacia el centro geométrico de objetos tridimensionales. Estas funciones son fundamentales en el ámbito de las estructuras de datos, ya que emplean algoritmos de búsqueda y optimización que son esenciales para resolver problemas de camino mínimo en grafos. Al modelar las mallas como grafos, estas operaciones optimizan la eficiencia en aplicaciones complejas como el renderizado en tiempo real, simulaciones físicas y animación, en ese sentido, se presentan comandos como ruta_corta y ruta_corta_centro, detallados en la subsección 4.1.

4.1 Descripción de los Comandos

Ruta_corta <i1> <i2> <nombre_objeto>

El comando *ruta_corta* permite identificar la ruta más corta entre dos vértices dados de un objeto tridimensional. Para que este comando funcione correctamente, es necesario que el objeto 3D esté cargado previamente en memoria; de lo contrario, el sistema notificará al usuario que el objeto no se encuentra disponible. Los índices de los vértices proporcionados (i1 e i2) deben ser válidos, es decir, deben encontrarse dentro del rango de vértices del objeto (entre 0 y n-1, siendo n el número total de vértices). Si ambos índices son iguales, se notificará al usuario que los vértices coinciden y no hay ruta que calcular.

El proceso comienza construyendo un grafo basado en la malla poligonal del objeto. Aquí, los vértices del grafo representan los puntos de la malla, y las aristas corresponden a las conexiones entre ellos. El algoritmo de Dijkstra se utiliza para calcular la ruta más corta entre los vértices i1 e i2, midiendo la distancia euclidiana entre los puntos. Este algoritmo garantiza que se encuentre la ruta con la menor distancia posible, utilizando los datos estructurados del objeto cargado.

Los resultados proporcionados por el comando incluyen una lista de los índices de los vértices que conforman la ruta óptima y la longitud total de esta ruta. Si no existe un camino que conecte los vértices, el sistema indicará que no se ha encontrado una

conexión posible. Para interpretar los resultados, el usuario debe revisar que los índices mostrados correspondan a una secuencia continua de vértices entre $i1$ e $i2$ y que la longitud de la ruta sea un valor finito y positivo. Si la distancia devuelta es infinita, esto sugiere que no hay un camino viable entre los vértices dados.

Además, para ilustrar el funcionamiento del comando, se puede considerar un ejemplo con un cubo unitario denominado Cubo1, compuesto por 8 vértices. Si se solicitan los vértices 0 y 6, el programa construirá el grafo del cubo y aplicará el algoritmo de Dijkstra, resultando en una ruta que pasa por los vértices 0, 1, 5 y 6, con una longitud total de 3.0. Esta salida puede verse ilustrada en la figura 66.

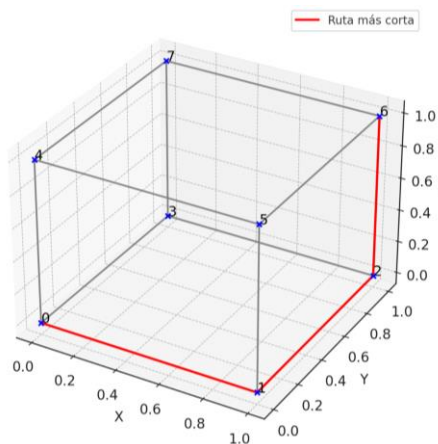


Figura 68. Ruta más corta de cubo unitario entre los vértices 0 y 6.

`Ruta_corta_centro <i1> <nombre_objeto>`

El comando `ruta_corta_centro` permite encontrar la ruta más corta desde un vértice especificado hasta el centro geométrico de un objeto tridimensional. Para que este comando funcione adecuadamente, es indispensable que el objeto 3D esté cargado en memoria; de lo contrario, el sistema notificará al usuario que el objeto no está disponible. El índice del vértice proporcionado ($i1$) debe ser válido, es decir, debe estar dentro del rango de vértices del objeto (de 0 a $n-1$, siendo n el número total de vértices). Si el índice está fuera de este rango, se informará al usuario que el índice proporcionado es inválido.

El proceso inicia construyendo un grafo basado en la malla poligonal del objeto. A partir de allí, se calcula el centro geométrico (centroide) del objeto, determinado por el promedio de las coordenadas de todos los vértices. Este punto central se agrega como un vértice adicional al grafo. Luego, se identifica el vértice más cercano al centroide y se conecta al grafo, facilitando la búsqueda de la ruta mínima. El algoritmo de Dijkstra

se emplea nuevamente para calcular la ruta más corta desde el vértice dado (i1) hasta el vértice que representa el centro geométrico.

Los resultados mostrados por el comando incluyen una lista con los índices de los vértices que conforman la ruta más corta y la distancia total de esta ruta. Si no existe un camino que conecte el vértice inicial con el centro del objeto, el sistema indicará que no se encontró una conexión posible. Para interpretar los resultados, el usuario debe verificar que los índices correspondan a una secuencia continua de vértices que conectan el vértice dado con el centro del objeto y que la distancia sea un valor finito y positivo. Si la distancia devuelta es infinita, indica que no hay un camino viable.

Como ejemplo, se utiliza un cubo para demostrar el funcionamiento del comando `ruta_corta_centro`, partiendo del vértice 2. El proceso comienza calculando el centroide del cubo, que se encuentra en la posición (5.5, 5.5, 5.5). Desde el vértice 2 (6, 6, 5), el algoritmo busca la ruta más corta hacia el centroide, considerando las conexiones del grafo. El camino óptimo identificado es $2 \rightarrow 1 \rightarrow 0$, ya que el vértice 1 (6, 5, 5) es el vecino más cercano al vértice 2, y a partir de ahí, el vértice 0 (5, 5, 5) es el siguiente punto de conexión. El vértice 0 es el más cercano al centroide, por lo que la ruta se completa sumando la distancia desde el vértice 2 hasta el vértice 0, seguida de la distancia euclidiana directa desde el vértice 0 hasta el centroide. Esta combinación garantiza que se minimice la distancia total recorrida, mostrando cómo el algoritmo aplica tanto la búsqueda en el grafo como el cálculo geométrico para determinar la ruta óptima.

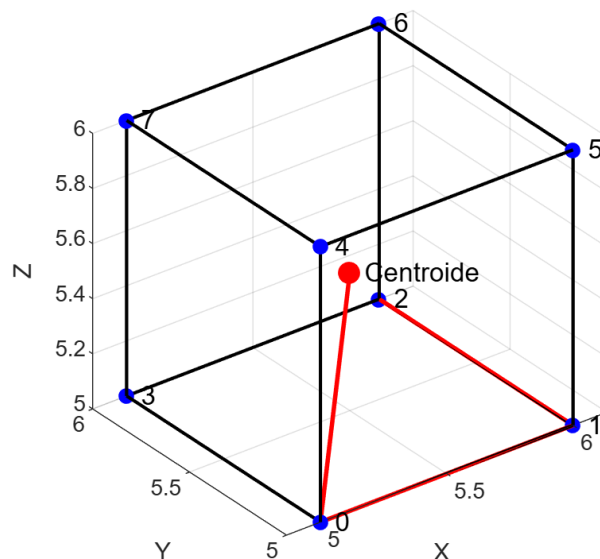


Figura 69: Ruta más corta desde vértice 0 a centroide.

4.2 Diseño

En esta subsección, se presentará el diseño detallado del sistema, el cual es de carácter acumulativo, reflejando fielmente su evolución a lo largo del proceso de desarrollo. A lo largo de este apartado, se describirán los Tipos Abstractos de Datos (TADs) empleados y las interacciones entre ellos, asegurando una representación precisa y completa del sistema en su estado actual.

Descripción de los Tipos Abstractos de Datos (TADs):

En los componentes anteriores del proyecto se definieron e implementaron los TADs: Objeto, Cara, Menú, Vértice, KDTree y NodoKDTree. Dado que estos TADs no han experimentado modificaciones en su diseño o funcionalidad, no se describirán nuevamente en este documento. La única excepción es el TAD Objeto, al cual se le ha añadido una lista de vértices para el almacenamiento de estos en el orden determinado por el archivo de texto.

En este tercer componente, se ha creado un nuevo TAD: Grafo, el cual se introdujo para representar de manera eficiente la conectividad entre los vértices de los objetos 3D. Este TAD permite implementar algoritmos para identificar rutas más cortas entre puntos específicos dentro de la malla, cumpliendo así con los requisitos de cálculo de rutas establecidos en la especificación del componente 3 del proyecto.

Grafo

El TAD Grafo es fundamental para la implementación de los comandos del componente 3 del sistema. Este TAD permite representar la estructura de vértices y aristas de un objeto 3D, facilitando la ejecución eficiente de algoritmos como Dijkstra para el cálculo de caminos mínimos entre puntos en el espacio tridimensional definidos por los vértices del grafo.

Estado:

- Vértices_del_grafo (vertices), vector de Vertices, lista de vértices que componen el grafo.
- matrizAdyacencia (matrizAdyacencia): matriz de números, matriz que representa las conexiones y costos entre los vértices.
 - Las entradas deben ser enteros representando el costo de conexión directa entre vértices o un valor máximo si no hay conexión directa.

Interfaz

- Grafo(objeto), Constructor que crea la matriz de adyacencia a partir de un objeto.

- Post: Inicializa vertices y matrizAdyacencia basándose en las caras del objeto, estableciendo conexiones directas entre vértices adyacentes.
- getNumVertices(): número, devuelve el número de vértices en el grafo.
 - Post: Retorna la cantidad de vértices como un entero.
- getVertices(): vertices, proporciona una lista de todos los vértices en el grafo.
- getMatrizAdyacencia():matrizAdyacencia, Devuelve la matriz de adyacencia del grafo.
- agregarVertice(vertice), Agrega un nuevo vértice al grafo y actualiza matriz de adyacencia.
- agregarArista(origen, destino): Agrega una conexión directa entre dos vértices, actualiza matrizAdyacencia con el costo calculado entre los vértices en origen y destino.
 - Pre: origen y destino deben ser índices válidos en vertices.
- Dijkstra(origen): Implementación del algoritmo de Dijkstra para calcular las distancias mínimas desde un vértice de origen a todos los demás.
 - Pre: origen debe ser un índice válido en vertices.
 - Post: Retorna un vector con las distancias mínimas desde origen.
- obtenerRuta(origen, destino): Reconstruye y devuelve la ruta más corta entre dos vértices.
 - Pre: origen y destino deben ser índices válidos en vertices y debe existir un camino entre ellos.

Objeto

El TAD objeto representa una entidad tridimensional completa que está compuesta por un conjunto de caras, cada una de las cuales está definida por sus vértices. Este TAD es esencial para modelar y manipular objetos completos en el espacio.

Estado

- Nombre_del_objeto (nombre), cadena de caracteres, identifica al objeto dentro del sistema.
 - nombre debe ser una cadena de caracteres válida, no puede haber otro objeto con ese nombre.
- Caras (caras), plantilla, contiene las caras que componen el objeto tridimensional.
- Vértices_totales (verticesTotales), vector de Vértices, contiene los vértices totales de la figura tridimensional en el orden preestablecido.

Interfaz

- Objeto(): objeto, crea un objeto por defecto sin nombre y sin caras.
- Objeto(nombre): objeto, crea un objeto con un nombre especificado, pero sin caras.
 - Pre: nombre debe ser una cadena válida, no puede haber otro objeto con ese mismo nombre.

- Objeto (nombre, caras): objeto, crea un objeto con un nombre y un conjunto de caras especificados.
 - Pre: nombre debe ser una cadena válida, no puede haber otro objeto con ese mismo nombre.
- getNombre(): nombre, retorna el nombre del objeto.
 - Pre: El objeto debe tener un nombre definido.
- getCaras(): caras, retorna las caras que componen el objeto.
 - Pre: El objeto debe tener caras definidas.
- getVerticesTotales(): verticesTotales, retorna los vértices totales de la figura tridimensional en el orden preestablecido.
- setNombre(nombre), establece un nuevo nombre para el objeto.
 - Pre: nombre debe ser una cadena válida; ningún otro objeto puede tener el mismo nombre.
- setCaras(caras), establece un nuevo conjunto de caras para el objeto.
- setVerticesTotales(vertices), establece un nuevo conjunto de vertices para el objeto.
- agregarCara(cara), añade una nueva cara a la lista de caras del objeto.
 - Pre: cara debe ser una cara válida.
- calcularCajaEnvolvente(xmin, ymin, zmin, xmax, ymax, zmax), calcula los límites de la caja envolvente que contiene al objeto.
 - Pre: El objeto debe tener vértices y caras definidas.
 - Post: Se calculan los límites mínimos y máximos en los ejes X, Y y Z que encierran el objeto.
- toString(): cadena de caracteres, retorna una representación textual del objeto, incluyendo su nombre y detalles de sus caras.

Diagrama de Relación de TADs:

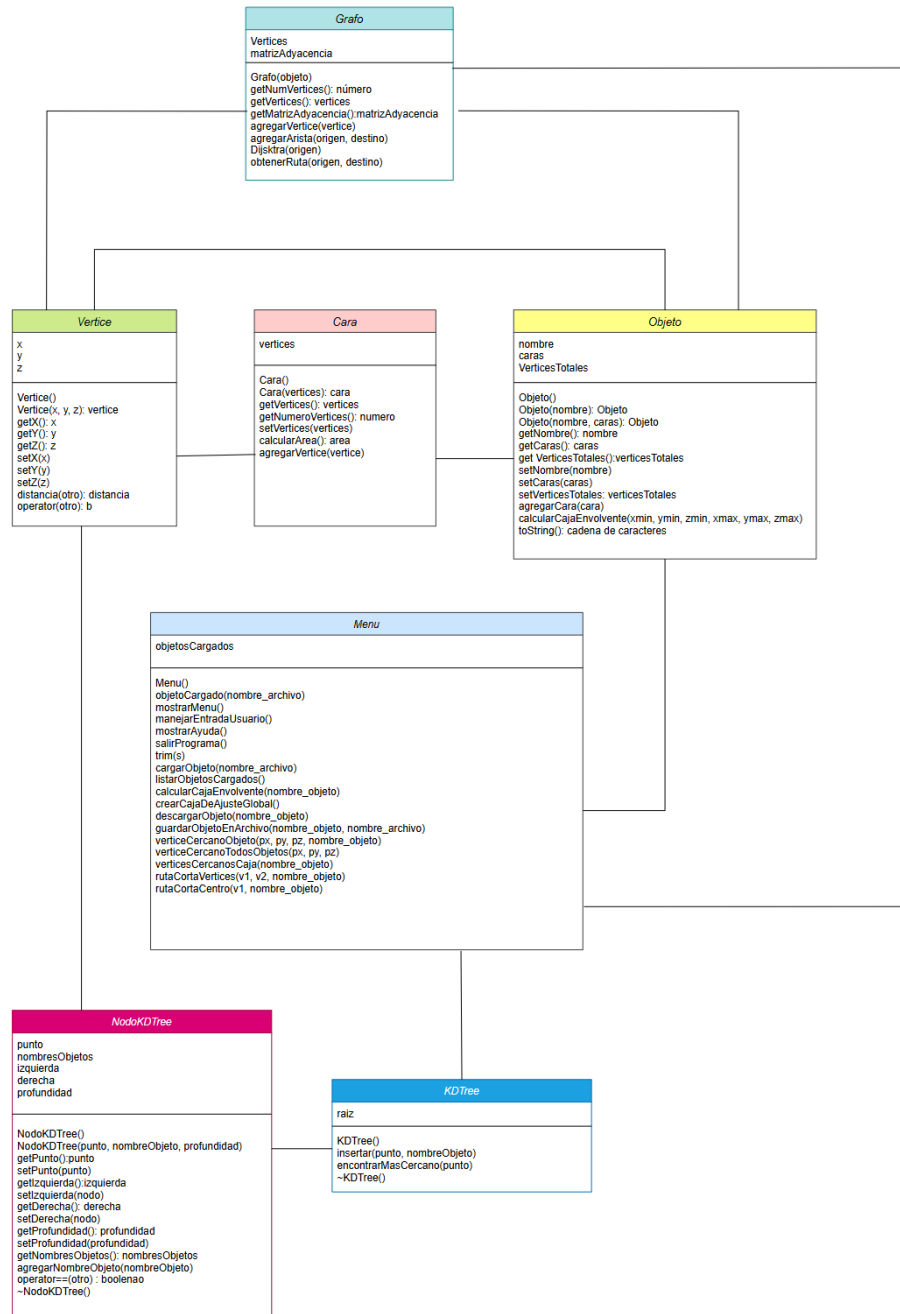


Figura 70. Diagrama de relación de TADs componente 3.

4.3 Plan de pruebas

En este conjunto de pruebas se evaluará la funcionalidad del comando `ruta_corta`, diseñado para identificar las rutas más cortas entre dos vértices dentro de un objeto 3D cargado en memoria. El objetivo principal del comando es calcular y mostrar la ruta más corta entre dos vértices dados, utilizando la distancia euclidiana entre ellos.

Para la primera prueba se va a tener en cuenta el cubo que se muestra a continuación:

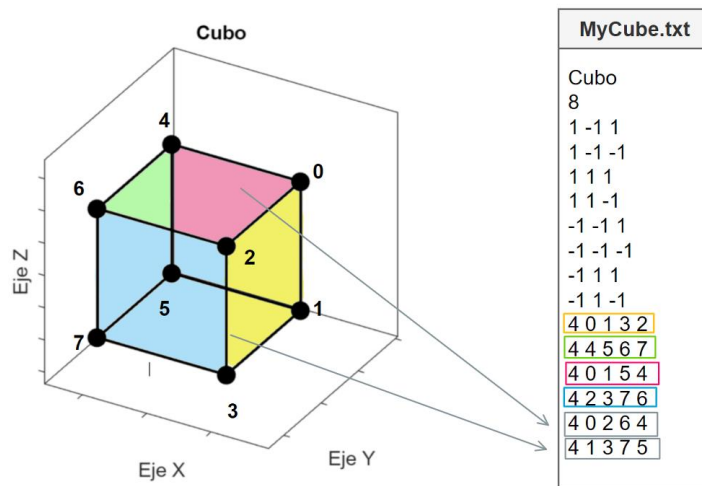


Figura 71. Cubo para la primera prueba.

El resultado del programa es el siguiente:

```
El comando 'ayuda' se ejecutó en 8.175e-05 segundos.
$ ruta_corta 5 2 Cubo
La ruta más corta que conecta los vértices 5 y 2 del objeto Cubo p
asa por: 5, 6, 2; con una longitud de 4.
El comando 'ruta_corta' se ejecutó en 2.638e-05 segundos.
```

Figura 72. Salida del programa.

Ruta entre los Vértices 5 y 2

Vamos a encontrar la ruta más corta entre los vértices 5 y 2.

1. Vértice 5: (-1, -1, 1)
2. Vértice 2: (1, -1, -1)

Conexiones entre los vértices:

- **Vértice 5** está conectado a **Vértice 6** (a través de la cara 3).
- **Vértice 6** está conectado a **Vértice 2** (a través de la cara 6).

Secuencia de Vértices:

La ruta más corta entre los vértices 5 y 2 es 5, 6, 2.

Verificación de la Distancia Euclidiana:

Vamos a calcular las distancias entre estos vértices utilizando la fórmula de distancia euclidiana en 3D.

- **Distancia entre 5 y 6:**

$$d_{5,6} = \sqrt{(-1 - (-1))^2 + (-1 - (-1))^2 + (1 - (-1))^2} = \sqrt{0 + 0 + 4} = \sqrt{4} = 2$$

Figura 73. Cálculo de la distancia entre los vértices 5 y 6.

- **Distancia entre 6 y 2:**

$$d_{6,2} = \sqrt{(-1 - 1)^2 + (-1 - (-1))^2 + (-1 - 1)^2} = \sqrt{(-2)^2 + 0 + (-2)^2} = \sqrt{8} \approx 2.828$$

Figura 74. Cálculo de la distancia entre los vértices 6 y 2.

- **Distancia total:**

$$d_{\text{total}} = d_{5,6} + d_{6,2} = 2 + 2.828 \approx 4.828$$

Figura 75. Cálculo de la distancia total.

A pesar de que en un cubo todas las aristas tienen la misma longitud, lo que implica que las distancias entre los vértices sean iguales, el algoritmo sigue siendo efectivo para identificar las rutas más cortas. Esto se debe a que el algoritmo recorre correctamente todas las posibles conexiones entre vértices, verificando las combinaciones adecuadas de aristas y asegurando que la ruta más corta se determine de manera precisa, incluso cuando las distancias son uniformes en toda la figura. El proceso de cálculo de distancias y la identificación de caminos se lleva a cabo correctamente, respetando la estructura del cubo.

Para la segunda prueba se usa un Tetraedro con la siguiente estructura:

1	Tetraedro
2	4
3	1 1 1
4	-1 -1 1
5	-1 1 -1
6	1 -1 -1
7	3 0 1 2
8	3 0 1 3
9	3 0 2 3
10	3 1 2 3
11	-1

Figura 76. Estructura del tetraedro a evaluar.

Gráficamente se vería de esta manera:

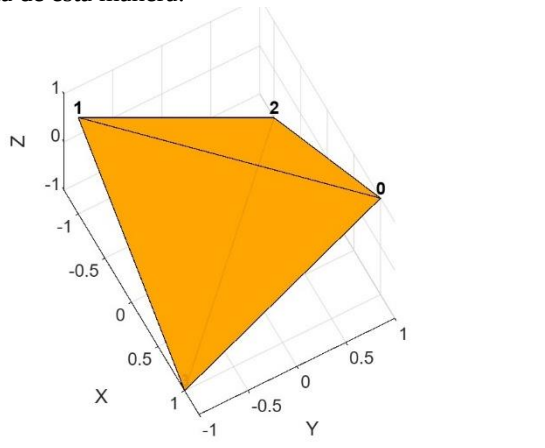


Figura 77. Imagen del tetraedro a evaluar.

Se evaluará la ruta más corta que conecta los vértices 1 y 3. El resultado del programa es el siguiente:

```
La ruta más corta que conecta los vértices 1 y 3 del objeto Tetraedro pasa por: 1, 3; con una longitud de 2.
```

Figura 78. Salida del programa

Esto se comprueba calculando la distancia euclidiana de los vértices 1 y 3 de la siguiente manera:

$$\begin{aligned}
 d &= \sqrt{(1 - (-1))^2 + (-1 - (-1))^2 + (-1 - 1)^2} \\
 d &= \sqrt{(1 + 1)^2 + (0)^2 + (-2)^2} \\
 d &= \sqrt{(2)^2 + (0)^2 + (-2)^2} \\
 d &= \sqrt{4 + 0 + 4} \\
 d &= \sqrt{8} \\
 d &= 2\sqrt{2} \approx 2.828
 \end{aligned}$$

Figura 79. Calculo de la distancia euclidiana de los vértices 1 y 3.

Dado que se está trabajando con un tetraedro y cada cara es un triángulo, la ruta más corta entre los vértices 1 y 3 será directamente a través de las aristas que los conectan.

En este caso, como ambos vértices están directamente conectados por una arista, la ruta más corta pasa solo por estos dos vértices: $1 \rightarrow 3$.

Como tercera prueba se hará uso de un prisma rectangular con la siguiente estructura:

```

1 Rectangular_Prism
2 8
3 0 0 0
4 10 0 0
5 10 1 0
6 0 1 0
7 0 0 1
8 10 0 1
9 10 1 1
10 0 1 1
11 4 0 1 2 3
12 4 4 5 6 7
13 4 3 2 6 7
14 4 0 1 5 4
15 4 0 3 7 4
16 4 1 2 6 5
17 -1

```

Figura 80. Estructura del prisma rectangular a evaluar.

Gráficamente se vería así:

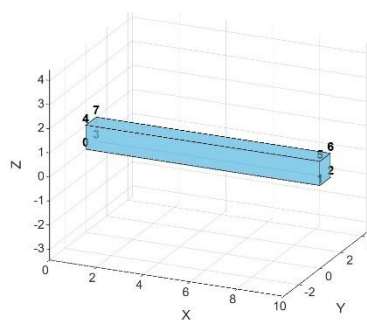


Figura 81. Gráfica del prisma rectangular.

Se calculará la ruta más corta entre los vértices 1 y 6. El resultado del programa es el siguiente:

```
$ ruta_corta 1 6 Rectangular_Prism
La ruta más corta que conecta los vértices 1 y 6 del objeto Rectan
gular_Prism pasa por: 1, 2, 6; con una longitud de 2.
El comando 'ruta_corta' se ejecutó en 3.03e-05 segundos.
```

Figura 82. Salida del programa.

1. **Distancia entre 1 y 2:**

- Vértice 1: (10, 0, 0)
- Vértice 2: (10, 1, 0)

Usando la fórmula de la distancia euclidiana:

$$d_1 = \sqrt{(10 - 10)^2 + (1 - 0)^2 + (0 - 0)^2} = \sqrt{1} = 1$$

Figura 83. Cálculo de la distancia euclidiana entre los vértices 1 y 2.

2. **Distancia entre 2 y 6:**

- Vértice 2: (10, 1, 0)
- Vértice 6: (10, 1, 1)

Usamos la fórmula de la distancia euclidiana:

$$d_2 = \sqrt{(10 - 10)^2 + (1 - 1)^2 + (1 - 0)^2} = \sqrt{1} = 1$$

Figura 84. Cálculo de la distancia euclidiana entre los vértices 2 y 6.

La distancia total es la suma de las distancias de los dos tramos:

$$d_{\text{total}} = d_1 + d_2 = 1 + 1 = 2$$

Figura 85. Suma total de las distancias.

La ruta más corta $1 \rightarrow 2 \rightarrow 6$ tiene una distancia total de 2, y esto coincide con lo que el programa calculó.

Este comportamiento es correcto, ya que las distancias entre los vértices en el cuboide son 1 en las direcciones de los ejes x, y o z, y el programa ha identificado el camino más corto a través de los vértices conectados por aristas.

5 Conclusiones

El proyecto aborda de manera integral la gestión y análisis de objetos tridimensionales representados mediante mallas poligonales. En la primera etapa, se implementaron funcionalidades para la organización y manipulación de objetos 3D, incluyendo la creación de cajas envolventes tanto para objetos individuales como para todos los objetos cargados en memoria. Este componente permitió delimitar espacialmente los objetos y facilitó su visualización y análisis. Las pruebas realizadas validaron el correcto manejo de diferentes escenarios, como objetos con coordenadas extremas, decimales y distribuidos en los octantes del espacio tridimensional, lo que confirma la robustez y fiabilidad del sistema.

En la segunda fase, el enfoque se centró en optimizar la búsqueda de vértices cercanos utilizando un KD-Tree, lo que mejoró la eficiencia al reducir la complejidad de búsqueda espacial. La implementación de esta estructura de datos permitió identificar rápidamente los vértices más cercanos, tanto dentro de un objeto específico como en relación a la caja envolvente. Las pruebas realizadas demostraron que el sistema maneja eficientemente grandes volúmenes de datos y situaciones complejas, incluyendo objetos detallados con múltiples vértices y caras.

En la última etapa del proyecto, se implementó un algoritmo para encontrar la ruta más corta entre dos vértices en los objetos tridimensionales, utilizando una representación basada en grafos. Se construyó un grafo donde cada vértice de la malla actúa como un nodo, y las aristas conectan los vértices adyacentes. Para calcular la ruta más corta, se utilizó el algoritmo de Dijkstra, conocido por su eficiencia en la búsqueda de caminos mínimos. El sistema fue sometido a pruebas exhaustivas, que confirmaron su precisión y capacidad para manejar estructuras complejas con múltiples vértices y caras. Esta implementación no solo mejoró la navegabilidad en las mallas 3D, sino que también destacó la versatilidad del enfoque aplicado, integrando algoritmos avanzados de teoría de grafos para resolver problemas complejos de manera eficiente.

Referencias

1. Filler, A. (2001). Dreidimensionale Computergrafik und Analytische Geometrie.
2. Gudalo, A. (2012). Polygonreduktion eines in der Half-Edge-Datenstruktur abge bildeten mit dem Silhouettenschnitt rekonstruierten 3D Objektes.
3. Bender, M., & Manfred, B. (2003). Computergrafik. Fachbuchhandlung beck.